



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

AUTOMATION DEPARTMENT

SISTEM AUTOMAT DE CONTROL ACCES PRIN DETECȚIE ȘI ANALIZĂ SPECTRALĂ A UNEI SECVENȚE AUDIO

PROIECT DE DIPLOMĂ

Autor: **Raluca-Diana TIMIȘ**

Conducător științific: **Prof. dr. ing. Ioan NAȘCU**

2026

Cuprins

1	INTRODUCERE	2
1.1	CONTEXT GENERAL	2
1.2	OBIECTIVELE LUCRĂRII	3
1.3	STRUCTURA LUCRĂRII	4
2	STUDIU BIBLIOGRAFIC	5
2.1	SISTEME EXISTENTE DE CONTROL ACCES.....	5
2.1.1	<i>Sisteme clasice și abordări bazate pe RFID / IoT.....</i>	<i>5</i>
2.1.2	<i>Sisteme de control al accesului bazate pe semnal audio.....</i>	<i>6</i>
2.2	FUNDAMENTE MATEMATICE	8
2.2.1	<i>Transformata Fourier Discretă (DTF).....</i>	<i>8</i>
2.2.2	<i>Algoritmul Transformatei Fourier Rapide (FFT).....</i>	<i>9</i>
2.2.3	<i>Teorema eșantionării (Nyquist-Shannon).....</i>	<i>9</i>
2.2.4	<i>Fereastra Hanning.....</i>	<i>10</i>
2.3	SPECIFICAȚIILE TEHNICE ALE COMPONENTELOR	11
2.3.1	<i>Arduino Uno</i>	<i>11</i>
2.3.2	<i>Microfon MAX9814.....</i>	<i>11</i>
2.3.3	<i>Servomotor MG995.....</i>	<i>12</i>
2.3.4	<i>Senzor ultrasonic HC-SR04</i>	<i>12</i>
2.3.5	<i>LED-uri și buzzer activ</i>	<i>12</i>
3	ANALIZĂ, PROIECTARE, IMPLEMENTARE	14
3.1	ARHITECTURA GENERALĂ A SISTEMULUI.....	14
3.2	IMPLEMENTARE HARDWARE.....	17
3.2.1	<i>Schema de conectare a componentelor.....</i>	<i>17</i>
3.2.2	<i>Considerații practice de montaj.....</i>	<i>21</i>
3.3	IMPLEMENTARE FIRMWARE ARDUINO.....	22
3.3.1	<i>Structura generală a firmware-ului</i>	<i>22</i>
3.3.2	<i>Mecanismul de eșantionare precisă</i>	<i>24</i>
3.3.3	<i>Protocolul de comunicație prin bloc binar</i>	<i>25</i>
3.3.4	<i>Controlul PWM manual al servomotorului</i>	<i>28</i>
3.3.5	<i>Logica de control a componentelor de semnalizare și proximitate</i>	<i>29</i>
3.4	IMPLEMENTARE SOFTWARE MATLAB	30
3.4.1	<i>Inițializare și conectare serială</i>	<i>30</i>
3.4.2	<i>Pipeline de procesare a semnalului.....</i>	<i>31</i>
3.4.3	<i>Analiza spectrală FFT</i>	<i>33</i>
3.4.4	<i>Algoritmul de detecție a secvenței.....</i>	<i>35</i>
3.4.5	<i>Interfața grafică</i>	<i>38</i>
3.5	REZULTATE EXPERIMENTALE	39
3.5.1	<i>Condiții de testare.....</i>	<i>39</i>
3.5.2	<i>Tabel rezultate</i>	<i>41</i>
3.5.3	<i>Analiza rezultatelor</i>	<i>43</i>
4	CONCLUZII.....	45
4.1	REZULTATE OBTINUTE	45
4.2	LIMITĂRI IDENTIFICATE	46
4.3	DIRECȚII DE DEZVOLTARE VIITOARE	47
5	BIBLIOGRAFIE	48

1 Introducere

1.1 Context general

Securizarea spațiilor și gestionarea controlului accesului prezintă o importanță semnificativă pentru administrarea clădirilor rezidențiale, a spațiilor industriale sau a instituțiilor publice. Soluțiile electronice sunt produsul evoluției tehnologice deoarece au capacitatea de a oferi jurnale de acces și un nivel de restricționare superior față de sistemele clasice, bazate pe chei fizice vulnerabile la pierdere sau duplicare neautorizată.

Piața sistemelor de acces din deceniile recente, în special cea din clădirile comerciale, a fost dominată de tehnologia de identificare prin radiofrecvență (RFID). Există, cu toate acestea, dezavantaje semnificative ale protocoalelor RFID standards precum limitările fizice legate de raza redusă de acțiune, sensibilitatea la interferențe electromagnetice provenite de la obiecte metalice, necriptarea cartelelor care pot fi clonate prin echipamente ușor accesibile din comerț, fiind compromisă securitatea fără a fi activată vreo alertă de securitate [1]. A urmat integrarea sistemelor Internet of Things, alternativă superioară tehnologiei RFID, care permit monitorizarea video și deplocarea ușii de la distanță. Arhitectura IoT prezintă și câteva vulnerabilități critice prin dependența de servere cloud externe și o infrastructură stabilă de rețea locală, orice defecțiune sau întrecupere a serviciilor furnizorului de cloud blochează temporar funcționalitatea sistemului de acces. De asemenea, prin continua expunere a modulului de control în mediul online, securitatea cibernetică a sistemului prezintă un risc constant de a fi atacată [2].

Explorarea unor metode alternative de securitate care are la bază procesarea locală, fără a fi dependentă de rețelele de date externe, devine necesară și este motivată direct de limitările și erorile specifice tehnologiilor RFID și IoT. Una dintre soluțiile tehnologice sigure și practice ar fi o abordare hibridă, unde este utilizat un canal acustic pentru a transmite informația de autentificare, prin implementarea unui sistem de detecție bazat pe analiză spectrală audio care funcționează la nivel local și elimină vulnerabilitățile cauzate de posibilele atacuri cibernetice asociate de conexiunile la internet. Un alt avantaj solid al acestei abordări este reprezentat de fezabilitatea crescută pe care o prezintă, folosind hardware aflat deja în posesia utilizatorilor, precum difuzoarele integrate ale dispozitivelor mobile, nefiind necesară achiziționarea și distribuirea unor cartele fizice dedicate.

1.2 Obiectivele lucrării

Lucrarea propune dezvoltarea, implementarea și validarea experimentală a unui prototip de control acces acționat acustic, cu o securitate autonomă prin analiza locală avansată a semnalelor digitale.

Proiectarea și implementarea unui sistem automat capabil să deblocheze un mecanism de acces, acționat de un servomotor, doar la detecția spectrală a unei secvențe temporale stricte, formată din trei secvențe audio pure (440 Hz, 550 Hz, 660 Hz), reprezintă principalul obiectiv al studiului.

Argumentele tehnice și matematice care stau la baza abordării acestei metode hibride de autentificare sunt următoarele:

- Complexitatea adăugată de secvența cu trei frecvențe: Sistemele bazate pe acoustic wake-up care folosesc un ton unic oferă avantaje privind reducerea consumului de energie, dar, în cazul lor pot apărea declanșări accidentale cauzate de zgomotul de fundal prin generarea a acelei frecvențe [3]. Astfel, succesiunea de trei frecvențe, a căror ordine trebuie respectată strict, crește complexitatea spațiului de parole, prevenind deschiderile accidentale sau încercările prin forță brută de efracție.
- Avantajul mediului acustic local: Procesarea undelor sonore este făcută direct în locația fizică a panoului de acces, independent de infrastructura de rețea, prevenind riscurile de întrerupere la distanță sau blocaje de rețea specifice arhitecturilor dependente de serverele cloud.
- Distribuția sarcinilor pe arhitectura MATLAB-Arduino: Sistemul propune o alocare distribuită a sarcinilor, din punct de vedere hardware, microcontrolerul Atmega328 al platformei Arduino Uno având o memorie SRAM limitată de doar 2 KB. Deoarece algoritmul folosește Transformata Fourier Rapidă (FFT) pe o fereastră de 512 puncte, cu precizie de virgulă mobilă, este necesară o rezoluție spectrală care să asigure o diferențiere corectă a tuturor tonurilor. Luând în calcul și coeficienții Hanning și factorii de rotație, ar fi nevoie de aproximativ 18 KB de memorie, rezultând într-un colaps al sistemului de achiziție (stack overflow). Pentru a fi asigurată o achiziție precisă și o validare eficientă a algoritmului de decizie, Arduino va fi utilizat doar pentru a prelua datele și a acționa motorul, în timp ce sarcinile de procesare matematică vor fi atribuite mediului de calcul MATLAB [4] [5].
- Robustețea oferită de analiza FFT în luarea deciziilor: Transformarea semnalului din domeniul timp în domeniul frecvenței prin FFT este mai stabilă la fluctuațiile în raportul semnal-zgomot, permițând o extragere mai precisă a tonului util din zgomotul ambiental, fiind aleasă în locul metodelor tradiționale precum detectarea vârfurilor (Peak Detection) sau metoda trecerilor prin zero (Zero Crossing) [6] [7].

Al doilea obiectiv major este constituit de demonstrarea capacității de rulare a unui sistem complet și stabil de prelucrare a semnalului digital, implicând preluarea unui vector de date, eliminarea componentei continue a microfonului, curățarea perturbațiilor electromagnetice inductive prin filtrarea statistică a acestora, aplicarea unei normalizări

dinamice pentru a menține independența între detecția semnalului și distanța la care se află sursa de emisie și multiplicarea semnalului cu o fereastră Hanning pentru a elimina, pe cât posibil, scurgerile spectrale, fiind pregătită o matrice finală pe care va fi aplicată transformarea Fourier.

Cercetarea este finalizată cu o validare experimentală a algoritmului în condiții reale de funcționare. Rata de detecție a prototipului va fi testată într-un spațiu acustic neizolat, secvența de frecvențe fiind captată prin intermediul unui modul microfon MAX9814 și controlând brațul mecanismului prin acționarea servomotorului MG995. Pentru a crește complexitatea proiectului, sistemul are în componență și alte componente suplimentare precum un senzor de proximitate HC-SR04 pentru a preveni închiderea barierei cât timp obiectul nu a trecut complet, LED-uri verde și roșu pentru semnalizarea vizuală a stării sistemului (acces acordat sau secvență invalidă) și un buzzer activ pentru feedback audio pe durata accesului acordat.

1.3 Structura lucrării

Capitolele care intră în componența structurală a lucrării reprezintă un ghid de introducere în fundamentele teoretice, implementarea sistemului, finalizată cu validarea cantitativă a rezultatelor fizice obținute.

Capitolul 2 prezintă un studiu bibliografic aprofundat al literaturii de specialitate care începe cu o analiză a metodelor actuale de control acces și a dezavantajelor pe care acestea le au, urmând cunoașterea arhitecturilor bazate pe comanda acustică. Ulterior, vor fi prezentate fundamentele matematice necesare procesării semnalelor prin Transformata Fourier Discretă (DFT), eficientizarea sa algoritmică prin Transformata Fourier Rapidă (FFT), necesitatea respectării teoremei Nyquist-Shannon și fenomenele spectrale diminuate prin aplicarea ferestrei Hanning, urmând o prezentare a specificațiilor componentelor hardware integrate în proiect.

În capitolul 3 este descris procesul tehnic de analiză, proiectare și implementare a sistemului cu o arhitectură hibridă de comunicare între mediul hardware și cel software. Este descrisă detaliat implementarea fizică a conexiunilor, structura firmware-ului C/C++ încărcat pe placa Arduino Uno pentru a prelua datele și genera impulsurile PWM manuale care controlează servomotorul MG995. Capitolul cuprinde explicația pipeline-ului de procesare a semnalului dezvoltat în mediul Matlab și logica matematică din spatele acesteia, fiind încheiat cu metodologia de testare a dispozitivului în mediul real și rezultatele experimentale obținute.

Capitolul 4 oferă concluziile lucrării prin evaluarea succesului îndeplinirii obiectivelor tehnice propuse inițial, menționarea limitărilor și a vulnerabilităților identificate și se încheie precizând direcțiile viitoare de dezvoltare și optimizare a platformei demonstrative.

2 Studiu bibliografic

Prima parte a acestui capitol introduce soluțiile clasice, cele bazate pe Internet of Things (IoT) și abordările acustice din domeniul sistemelor de control al accesului, fiind o analiză amănunțită asupra actualității tehnologice a acestui domeniu. În cea de-a doua parte, vor fi prezentate fundamentele matematice ale prelucrării semnalelor digital, punând în evidență a tranformatelor Fourier, tehnicilor de ferestruire, împreună cu specificațiile tehnice al echipamentului hardware folosit pentru dezvoltarea sistemului propus.

2.1 Sisteme existente de control acces

2.1.1 Sisteme clasice și abordări bazate pe RFID / IoT

La început au fost folosite încuietorile mecanice pentru a restricționa accesul și a asigura securitatea spațiilor. Fiind utilizate doar cheile fizice, furtul, pierderea sau defecțiunile mecanice, copiile făcute de către persoanele neautorizate reprezentau riscuri majore. Monitorizarea accesului era imposibilă din cauza lipsei unui jurnal de acces. Astfel, au fost implementate codurile PIN la tastatură numerică, o primă evoluție tehnologică în acest domeniu. Vulnerabilitățile existau și în acest caz, uzura fizică inevitabilă a butoanelor în timp sau spionarea directă prin metoda de shoulder surfing, dezvăluind cifrele din componența parolei.

Următorul pas din dezvoltarea acestui domeniu consta în sistemele de acces electronice bazate pe identificare prin radiofrecvență (RFID). Această metodă elimina majoritatea limitărilor metodelor tradiționale, permițând recunoașterea utilizatorului și stocarea datelor de acces. Dezavantajele principale erau condiționările impuse de mediul fizic de propagare, senzorii având o rază de citire limitată, între 0 și 20 mm [1]. În plus, prezența obiectelor metalice dintre senzor și cartelă putea bloca complet semnalul, sistemul fiind sensibil la interferențe [1]. Clonarea cartelelor RFID cu ajutorul dispozitivelor comercializate la prețuri reduse putea compromite securitatea spațiului, pe lângă constângerile structurale [1].

Sistemele de control acces s-au modernizat datorită implementării bazate pe arhitectura Internet of Things (IoT), care au în componență aplicații mobile dedicate sau module de achiziție video precum ESP32-CAM. Aceste dispozitive permit deblocarea ușii de la distanță și supravegherea spațiului prin imagini transmise în timp real, fiind trimise alerte sonore în cazul tentativelor de intrare prin efracție, putând fi verificată identitatea persoanelor care accesează zona [2]. Dependența de o conexiune stabilă la Wi-Fi sau date mobile și de continua funcționare a serverelor cloud ale furnizorului unde orice eroare a infrastructurii poate restricționa accesul și bloca poarta [2].

În ciuda capacităților avansate oferite de sistemele IoT, atât expunerea la atacurile cibernetice din mediul online, cât și nevoia de o conexiune continuă și stabilă la internet,

influențează semnificativ fiabilitatea sistemului. Constrângerile tehnologiilor RFID și IoT conduc la necesitatea explorării unor variante de acces care nu depind de infrastructura de rețea, bazate pe procesare locală.

2.1.2 Sisteme de control al accesului bazate pe semnal audio

O soluție care să nu fie dependentă de conexiunea la internet și care să utilizeze difuzorul unui smartphone sau vocea umană, dispozitive deținute deja de utilizatori, o reprezintă sistemele de securitate bazate pe semnalele acustice. Având fiecare avantaje hardware sau matematice sau dezavantaje de design, literatura de specialitate introduce trei mari categorii.

Sistemele de blocare bazate pe tehnologia DTMF (Dual-Tone Multi-Frequency) reprezintă prima categorie clasică, implementată în automatizarea locuințelor. Prin folosirea unui modul decodor hardware DTMF și integrată pe o plăcuță Arduino Uno, H. A. Rangkuti și J. W. Simatupang au dezvoltat o arhitectură automatizată de deschidere a ușilor [4]. Sistemul are la bază semnalele polifonice standard generate prin apăsarea tastaturii virtuale a unui telefon mobil. În regimul de așteptare (standby mode) este rulată o parolă formată din 4 digiți de către sistem. Dacă secvența corectă a fost introdusă, este realizat un mecanism automat de protecție prin revenire prin generarea, printr-un buzzer, unui semnal de validare de către microcontroler care cuplează un releu electromagnetic și retrage tija unui selenoid de blocare pe durata a 3 secunde [4].

Limitările razei de funcționare stabilă a acestui sistem reies din datele oferite de autori în urma testărilor, distanța geometrică fiind între minim 1 și maxim 10 cm [4]. Se observă o capacitate de captare extrem de redusă deoarece, la distanțe peste 10 cm, din cauza atenuării unde acustice în aer, sistemul nu oferă niciun răspuns. Un alt lucru care dezavantajează sistemul este reprezentat de securitatea scăzută de care dă dovadă, în ciuda posibilității de schimbare a parolei prin apăsarea tastei * (asociată valorii -2 în monitorul serial), utilizatorul fiind avertizat acustic de trei ori cu un delay de 100 ms pentru introducerea vechiului și noului cod. Blocarea temporară sau permanentă a controlerului nu este activată de tastarea repetată unei parole incorecte, spre deosebire de sistemele bancare de tip ATM, ceea ce face sistemul expus la încercări repetate de acces în sistem [4].

O a doua perspectivă a fost abordată de către Z. Hu, Y. Han și N. Ahmen care au dezvoltat un modul de trezire acustică (acoustic wake-up) de complexitate joasă, cu scopul de a îmbunătății algoritmi de detecție pentru sistemele izolate și de a optimiza consumul energetic [3]. Acest sistem era destinat controlului echipamentelor standalone care se folosesc de energie limitată. Deoarece, de obicei, bateriile echipamentelor se descarcă repede din cauza utilizării unor secvențe pseudo-aleatoare care au nevoie de o putere computațională mare din partea cipului de procesare și de circuite analogice sofisticate sau a unor semnalizări complexe de tip modulație liniară în frecvență (LFM), autorii lucrării au venit cu ideea unui sistem care utilizează un stimul simplu, format dintr-un impuls sinusoidal pur de o durată de 100 ms, cu o frecvență fixă de 12 kHz, rezolvând problema consumului ridicat [3].

Optimizarea propusă de autori presupunea criteriul decizional aplicat în faza de analiză spectrală, din punct de vedere matematic. După eșantionarea semnalului acustic

și aplicarea algoritmului Transformatei Fourier Rapide (FFT), se adună componentele din spectrul de amplitudine, urmărindu-se depășirea unui prag critic impus pentru evaluare, conform relației standard [3]:

$$\sum_{i=0}^{W-1} X(k_0 + i) \geq \gamma \quad (2.1)$$

$X(k_0)$ reprezintă magnitudinea la frecvența țintă, W este lățimea de bandă alocată ferestrei de analiză iar γ constituie pragul de decizie ales [3]. Pentru a scăpa de extragerea rădăcinii pătrate din calcule impuse pentru a determina modulul în magnitudine, cercetătorii l-au înlocuit cu o evaluare directă a spectrului de putere ($X^2(k)$) [3]. Acest lucru a fost făcut deoarece radicalul, din punct de vedere computațional, reprezintă o operație complexă și problematică pentru arhitecturile hardware de putere mică. Ambele metode obțin rezultate identice, conform analizei comparative de performanță a acestora pentru un raport semnal-zgomot (SNR) de până la -6 dB, sub acest prag diferența de doar 1 dB fiind nesemnificativă. Consumul mediu a fost redus la valoarea de 7.2 mW prin această metodă (față de 100 mW observat în metodele standard) și s-a menținut o detecție cu probabilitate de succes de peste 90% [3].

Cea de-a treia categorie din stadiul actual urmărea eliminarea necesității de generare a unor tonuri artificiale și sporea flexibilitatea, fiind introdusă de către R. R. Raj și colegii săi. Metoda alternativă utiliza procesarea biometrică a vocii umane, implementând un sistem de acces care utilizează interfața directă dintre mediul de calcul numeric MATLAB și platforma hardware Arduino [5]. Prin aplicarea algoritmului Transformatei Fourier Rapide (FFT) era realizată prelucrarea semnalului vocal captat în timp real de microfon, reușind să se izoleze și extragă caracteristica vocală umană fundamentală, adică înălțimea sunetului (pitch) [5].

Pentru a obține o valoare numerică exactă a înălțimii sunetului, algoritmul FFT generează o reprezentare spectrală a undei de intrare, exemplul din studiu fiind referința de 1628 pentru un eșantion înregistrat [5]. Sistemul este controlat astfel: pentru a retrage încuietoarea, motorul se acționează și se rotește în sens trigonometric dacă comanda rostită este „open the door”, iar, pentru a încuia ușa, comanda rostită trebuie să fie „close the door”, motorul executând o rotație în sensul acelor de ceasornic [5]. Prin urmare, dacă spectrul vocal introdus se potrivește cu datele din MATLAB, este trimis un semnal serial către plăcuța Arduino pentru a acționa motorul. Fiind comparat cu metodele standard bazate pe Transformata Cosinus Discretă (DCT), algoritmul bazat pe FFT înregistrează cu succes o performanță mult mai ridicată cu o acuratețe de clasificare spectrală de 76%, față de DCT care prezintă o acuratețe de doar 56% [5]. Ceea ce dezavantajează această variantă este, însă, instabilitatea spectrală a vocii umane care poate fi afectată de factori fiziologici precum oboseală sau răgușeală, nereușind să fie recunoscută de sistemul de detecție, precum și sensibilitatea la zgomotul de fundal.

Din cauza acestor vulnerabilități, raza redusă a sistemelor DTMF, dependența energetică a modulelor wake-up și instabilitatea biometrică a recunoașterii vocale din lucrările analizate, este necesară abordarea de noi variante. Lucrarea de față propune

detecția spectrală a unei secvențe de trei frecvențe prin FFT, cu procesare locală și independentă de rețea.

2.2 Fundamente matematice

2.2.1 Transformata Fourier Discretă (DTF)

Pentru a putea identifica existența unor frecvențe specifice este nevoie ca semnalul primit să fie trecut din domeniu timp în domeniul frecvență, deoarece, în sistemele digitale bazate pe recunoaștere acustică, semnalul captat de microfon este convertit într-o secvență de valori numerice discrete. Instrumentul care stă la baza procesării semnalelor digitale este Transformata Fourier Discretă (DFT) [7].

Spectrul de frecvență $X(k)$ pentru o secvență finită $x(n)$ de lungime N , se obține prin aplicarea următoarei formule de definiție [8]:

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, \quad k = 0, 1, \dots, N-1 \quad (2.2)$$

unde factorul exponențial complex, denumit și factor de fază (twiddle factor), este definit prin relația [8]:

$$W_N = e^{-j\frac{2\pi}{N}} \quad (2.3)$$

Există două proprietăți matematice importante ale DTF puse în evidență de teoria prelucrării semnalelor și anume liniaritatea și periodicitatea [7]. Liniaritatea demonstrează faptul că scalarea și adunarea semnalelor se păstrează în transformare și este definită de relația [7]:

$$DFT\{ax_1(n) + bx_2(n)\} = aX_1(k) + bX_2(k) \quad (2.4)$$

Proprietatea de liniaritate a transformatei asigură că nu vor fi generate componente de intermodulație de către zgomotul din mediu sau interfețele spectrale. Astfel, deși validarea parolei are la bază identificarea unei succesiuni secvențiale de tonuri, evitând suprapunerea simultană a acestora, fiecare frecvență de interes este izolată corect [7].

Proprietatea de periodicitate este exprimată matematic sub forma [7]:

$$X(k + N) = X(k) \quad (2.5)$$

Spectrul calculat pe intervalul fundamental $0 \leq k \leq N-1$ înregistrează toată informația spectrală necesară, evaluarea sa peste acest interval fiind una inutilă [7]. Astfel, ne este demonstrat faptul că Transformata Fourier Discretă este periodică, cu o perioadă egală cu numărul total de eșantioane N dintr-o fereastră de analiză.

Calculul DFT prezintă o problemă semnificativă de scalabilitate, în ciuda bazei matematice solide oferite de DFT, algoritmul având nevoie de un număr total de operații proporțional cu N^2 pentru a calcula tot spectrul de N puncte [8]. Această complexitate de ordinul $O(N^2)$ este un obstacol critic în dezvoltarea sistemelor embedded. Din cauza faptului că un microcontroler de tipul Atmega328 al platformei Arduino Uno prezintă limitări în resursele de memorie și în puterea de procesare, ecuația DFT clasică consumă

un volum foarte mare de timp și energie, fiind imposibilă analiza spectrală în timp real a parolelor acustice și impunând folosirea unui algoritm mai optimizat [8].

2.2.2 Algoritmul Transformatei Fourier Rapide (FFT)

Metoda Transformatei Fourier Rapide (FFT) reduce complexitatea computațională de ordinul $O(N^2)$ specifică DFT, motiv pentru care este utilizată în implementările practice, optimizând procesul prin descompunerea iterativă a unei secvențe de lungime N în subsecvențe de dimensiuni mai mici [8]. Strategia de tip decimare în timp Cooley-Tukey împarte secvența discretă de intrare în eșantioane cu indecși pari și eșantioane cu indecși impari, fiind generate două blocuri distincte de lungime $N/2$ [8]. Acest proces continuă recursiv pentru a se ajunge la transformate elementare de un singur punct [8]. Numărul total de operații aritmetice este redus la un ordin de complexitate de $O(N \log_2 N)$, prin reconstruirea rezultatelor parțiale în spectrul final de lungime N , prin intermediul factorilor de rotație [8]. Este permisă, astfel, optimizarea procesării semnalelor în timp real fără a fi necesare resurse excesive ale sistemului de calcul.

Precizia detecției și imunitatea la zgomot în raport cu celelalte tehnici de analiză a frecvenței în domeniu timp, cum ar fi detecția vârfurilor (Peak Detection) sau metoda trecerii prin zero (Zero Crossing), reprezintă două alte avantaje importante pe care le oferă FFT, pe lângă eficiența computațională [6]. Detecția vârfurilor identifică valorile maxime locale pentru a calcula perioada undeii, iar metoda trecerii prin zero calculează frecvența prin contorizarea intersecțiilor semnalului cu axa nulă [6]. Aceste tehnici au o sensibilitate foarte mare la zgomotul de fundal și la fluctuațiile mici de amplitudine, rezultând în erori mari în mediile reale, în ciuda rapidității și a memoriei minime de care au nevoie [6]. Se poate aplica metoda de completare cu zerouri (zero-padding) pentru îmbunătățirea rezoluției în frecvență și observarea mai clară a spectrului, fără modificarea duratei de achiziție a semnalului, extinzând secvența de intrare până la dimensiunea aleasă a ferestrei FFT [6].

Sistemul propus dorește utilizarea algoritmului FFT în locul metodelor în domeniul timp ZC (Zero Crossing) și PD (Peak Detection) deoarece FFT menține o deviație standard redusă a erorii, cu o robustețe ridicată la variațiile raportului semnal-zgomot (SNR) scăzut, fapt ce poate fi observat în lucrarea realizată de M. M. Jazini și echipa sa [6]. Analiza datelor cantitative prin FFT evită declanșările false în prezența zgomotului ambiant, izolând cu succes componentele utile în canale spectrale separate [6].

2.2.3 Teorema eșantionării (Nyquist-Shannon)

Teorema eșantionării Nyquist-Shannon stă la baza conversiei semnalului acustic continuu captat de modulul microfonului într-o secvență numerică discretă realizată de un convertor analog-digital (ADC) [7]. Teorema demonstrează faptul că un semnal continuu limitat în bandă, cu o frecvență maximă notată cu $F_{\max}$, poate fi reconstruită exact din eșantioanele sale discrete dacă și numai dacă frecvența de eșantionare F_s îndeplinește condiția [7]:

$$F_s \geq 2F_{\max} \quad (2.6)$$

Pragul minim $2F_{\max}$ reprezintă rata Nyquist, fiind limita critică sub care procesul de digitalizare prezintă perturbații ireversibile în semnalul rezultat [7].

Fenomenul de aliasing, sau suprapunere spectrală, apare la nerespectarea acestei condiții ($F_s \geq 2F_{max}$), caz în care componentele de frecvență înaltă depășesc limita Nyquist și se reflectă greșit în banda de interes [9]. Astfel, acestea se suprapun peste frecvențele joase și fac separarea și identificarea corectă a componentelor spectrale prin FFT imposibilă [9]. Proiectul propus folosește o secvență audio compusă din trei tonuri pure succesive, frecvențele fiind de 440 Hz, 550 Hz și 660 Hz. Frecvența maximă (F_{max}), cea de 660 Hz, impune o frecvență minimă teoretică de eșantionare de 1320 Hz [7]. Pentru ca distorsiunile spectrale să fie evitate, iar toleranța, optimă, frecvența reală de eșantionare a sistemului (F_s) are valoarea de 1429 Hz, depășind pragul minim teoretic și oferind o bază stabilă pe 512 puncte pentru analiza spectrală.

2.2.4 Fereastra Hanning

Multiplicarea semnalului cu o fereastră dreptunghiulară temporară este, din punct de vedere matematic, echivalentul pentru achiziția unui bloc finit de eșantioane ($N=512$) dintr-un semnal acustic continuu [9]. Operația de tăiere bruscă a semnalului introduce discontinuități pronunțate la capetele intervalului de analiză, în domeniul frecvență fiind tradusă prin convoluția spectrului ideal al semnalului cu funcția de tip sinus cardinal (sinc), caracteristică ferestrei dreptunghiulare, care generează lobi secundari accentuați [9]. Fenomenul cunoscut sub numele de scurgere spectrală (spectral leakage) determină disiparea energiei unei frecvențe pure în benzile de frecvență adiacente, mascând componentele utile și alterând amplitudinile reale ale semnalului [9].

Semnalul eșantionat este multiplicat, prin urmare, cu o fereastră ponderată numeric ale cărei margini coboară ușor spre valoarea zero, înainte de aplicarea FFT, diminuând efectele scurgerilor spectrale [9]. S-a optat pentru utilizarea ferestrei Hanning, cunoscută și sub numele de fereastră Cosinus Ridicat, funcția sa de ponderare pentru o secvență cu lungimea N fiind definită matematic prin relația [9]:

$$w(n) = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right), \quad 0 \leq n \leq N-1 \quad (2.7)$$

Fereastra dreptunghiulară standard atenuază cu -13.3 dB doar primul lob secundar, ceea ce permite o scurgere spectrală într-o rată semnificativă de 9.14% din energia totală a semnalului, în timp ce, aplicarea ferestrei Hanning își demonstrează superioritatea tehnică prin modificarea comportamentului spectral, asigurând o atenuare a lobilor secundari mult mai mare, de -42.5 dB, reducând scurgerea totală la o valoare reziduală de 0.03% [7].

Utilizarea unei simple ferestre dreptunghiulare ar împrăști energia pe zeci de canale adiacente, depășind pragul de zgomot și generând erori de detecție în cazul unui ton generat care nu coincide exact cu centrul unui bin spectral. Alegerea ferestrei Hanning a fost influențată, deci, de avantajele aduse în proiect, fiind o soluție optimă pentru pre-procesarea semnalului, asigurând stabilitate decizională pentru algoritmul de control acces și asigurând încadrarea micilor fluctuații în intervalul de toleranță stabilit.

2.3 Specificațiile tehnice ale componentelor

2.3.1 Arduino Uno

Platforma de dezvoltare Arduino Uno, bazată pe microcontrolerul Atmega328 de 8 biți, memorie SRAM de 2 KB, memorie Flash de 32 KB pentru stocarea codului executabil și o memorie EEPROM de 1 KB, reprezintă unitatea de bază pentru achiziția de date și controlul local din structura proiectului [10]. Frecvența de ceas la care rulează microcontrolerul este de 16 MHz, integrând un convertor analog-digital (ADC) intern pe 10 biți, capabil să convertească tensiunile analogice de intrare în valori numerice cuprinde în intervalul 0 – 1023 [10]. Viteza cu care se desfășoară comunicația cu sistemul de calcul prin intermediul interfeței UART este una ridicată de 115200 baud, fiind utilizat un protocol de transmisie în bloc binar prin markeri specifici.

Interfața directă MATLAB – Arduino este una utilizată des în proiectele de cercetare cu complexitate similară, precum în studiul propus de Raj, unde este folosită pentru a transmite comenzi de acționare după clasificarea spectrală a semnalelor vocale [5]. Fiabilitatea și flexibilitatea acestei arhitecturi hardware în aplicațiile de control standalone este evidențiată și în structura de automatizare dezvoltată de Rangkuti, microcontrolerul Atmega328 stând la baza deciziilor de decodare și validare ale secvențelor de acces [4].

Există limitări stricte, în cazul acestei arhitecturi, cauzate de cei 2 KB SRAM ai microcontrolerului Atmega328, producându-se depășirea memoriei prin rularea locală a algoritmului FFT pe 512 puncte cu precizie, fiind necesari aproximativ 18 KB. Pentru a asigura funcționarea corectă a sistemului, pipeline-ul de procesare spectrală este distribuit pentru a rula în mediul MATLAB pe laptop, placa Arduino fiind utilizată exclusiv pentru achiziția binară de date și trimiterea comenzilor locale.

2.3.2 Microfon MAX9814

Rolul modulului de microfon bazat pe cipul MAX9814 este de a capta a undele acustice ambientale și a le transforma în semnale electronice variabile în timp, având integrat un microfon cu condensator electret de înaltă sensibilitate și un amplificator cu control automat al câștigului (Automatic Gain Control, sau AGC), funcție esențială în proiectul curent [11]. Cu ajutorul acestuia fenomenul de saturare sau tăiere a semnalului (clipping), care apare atunci când sursa audio este aflată aproape de microfon, este prevenit, menținându-se și un timp de câștig ridicat pentru semnalele acustice slabe sau de la distanță.

Alimentarea modulului se face la o tensiune continuă de 3.3 V sau 5 V și produce la ieșire un semnal analogic de aproximativ 1.25V, centrat în jurul unei tensiuni de polarizare (bias), volumul adaptându-se în funcție de nivelul zgomotului de fond [11]. Banda ocupată de răspunsul în frecvență al microfonului este cuprinsă între 20 Hz și 20 kHz, având o acuratețe optimă aflată în intervalul tonurilor pure alese pentru parola acustică, iar ieșirea analogică a modulului este conectată direct la pinul A0 al convertorului ADC de pe placa Arduino Uno pentru a permite ieșirea periodică stabilă de 700 μs a formei de undă, digitalizarea fiind realizată într-un mod stabil.

2.3.3 Servomotor MG995

Caracteristicile principale ale servomotorului de mare cuplu model MG995 care se ocupă de simularea deschiderii, respectiv blocării căii de acces sunt rezistența mecanică sporită la uzură și un cuplu de blocare ridicat, de aproximativ $10 \text{ kg} \cdot \text{cm}$ la o alimentare de 4.8 V și de până la $12 \text{ kg} \cdot \text{cm}$ la o tensiune de 6 V , fiind echipat cu angrenaje metalice complete [12]. Axul are o viteză de rotație de 0.20 secunde pentru 60 de grade, asigurând o retragere rapidă a tijei de blocare la primirea unei parole audio corecte.

Servomotorul este operat pe baza unui semnal de comanda de tip modulație în lățime a impulsurilor (PWM) cu o frecvență standard de 50 Hz pe o perioadă de 20 ms . Unghiul de orientare al axului este acționat prin generarea manuală a impulsurilor de sincronizare direct din codul microcontrolerului Arduino, fiind trimis un tren de 50 de pulsuri compuse dintr-o stare logică înaltă cu durata de $1500 \mu\text{s}$, urmată de o stare joasă de $19000 \mu\text{s}$. Controlul manual al servomotorului elimină utilizarea librăriei sale software complexe a cărui temporizator bazat pe întreruperi producea erori semnificative ale eșantionării prin efectul său de jitter, în contact cu magistrala URAT, fiind evitat și un algoritm de control bazat pe un model matematic explicit. Prin urmare, servomotorul este tratat strict ca un element de execuție în arhitectura proiectului propus.

2.3.4 Senzor ultrasonic HC-SR04

Detectarea prezenței unui obiect în zona barierei și prevenirea închiderii premature a acesteia sunt realizate prin intermediul modulului senzor ultrasonic HC-SR04. Acesta asigură măsurarea distanței față de obstacole prin metoda eco-locăției, fără contact fizic, în intervalul de $2 \text{ cm} - 400 \text{ cm}$, cu o acuratețe de până la 3 mm [13].

Principiul de funcționare se bazează pe emisia unui tren de opt impulsuri ultrasonice la frecvența de 40 kHz , declanșat printr-un semnal TTL de nivel înalt cu durata minimă de $10 \mu\text{s}$ aplicat pe pinul Trig [13]. Modulul detectează semnalul reflectat de obstacol și generează pe pinul Echo un impuls de nivel înalt a cărui durată este proporțională cu timpul de propagare dus-întors al undei ultrasonice. Distanța față de obiect se calculează conform relației [13]:

$$d = t \cdot 340 / 2 \text{ [cm]} \quad (2.8)$$

unde t reprezintă durata impulsului Echo exprimată în microsecunde, iar 340 m/s este viteza sunetului în aer. În implementarea firmware, această relație este calculată prin expresia echivalentă $\text{duration} \times 0.034 / 2.0$.

Modulul funcționează la o tensiune de alimentare de 5 V curent continuu, consumând un curent de 15 mA , și prezintă un unghi de măsurare de 15 grade [13]. Datorită curentului de funcționare, alimentarea senzorului se realizează din sursa externă MB102, și nu direct din pinul de 5 V al microcontrolerului Arduino, pentru a evita supraîncărcarea regulatorului de tensiune al acestuia. În cadrul sistemului, pinul Trig este conectat la pinul digital 5, iar pinul Echo la pinul digital 6 al platformei Arduino Uno.

2.3.5 LED-uri și buzzer activ

Semnalizarea vizuală și acustică a stării sistemului este realizată prin două diode electroluminescente (LED) și un buzzer activ. LED-ul verde, conectat la pinul digital 4

printr-o rezistență de limitare a curentului de 220 Ω , se aprinde pe durata acordării accesului, confirmând vizual starea ACCESS GRANTED. LED-ul roșu, conectat la pinul digital 3 printr-o rezistență identică de 220 Ω , emite patru clipiri rapide la recepționarea comenzii 'T' de la MATLAB, semnalizând fie expirarea ferestrei temporale (timeout), fie o respingere activă a secvenței introduse.

Buzzerul activ, conectat direct la pinul digital 7 fără rezistență externă, emite semnale acustice ritmice cu durata de 80 ms la intervale de 500 ms pe durata menținerii barierei deschise, oferind un feedback sonor continuu utilizatorului care tranzitează punctul de acces. Spre deosebire de buzzerul pasiv, cel activ integrează circuitul oscilator intern, generând tonul la simpla aplicare a tensiunii de alimentare, fără a necesita un semnal PWM extern.

3 Analiză, proiectare, implementare

3.1 Arhitectura generală a sistemului

Pentru a obține un echilibru între achiziția de date în timp real și execuția unor algoritmi complecși de procesare digitală a semnalelor (DSP), în proiectarea și dezvoltarea sistemului automat de control al accesului, bazat pe recunoașterea și analiza spectrală a unei secvențe audio prestabilite a fost folosită o arhitectură generală hibridă, sub forma unei topologii distribuite, care să satisfacă cele două cerințe. Astfel, sistemul separă la nivel fizic și logic sarcinile critice în timp, captarea eșantioanelor și controlul actuatorului mecanic, de cele care consumă mai multe resurse computaționale, adică filtrarea semnalului, analiza sa spectrală și luarea deciziilor. Cele două noduri operaționale în care a fost divizat sistemul sunt nodul periferic de achiziție și execuție, implementat pe platforma hardware Arduino Uno, și unitatea centrală de procesare numerică, implementată în mediul software MATLAB, rulat pe un sistem de calcul gazdă (laptop). În ce privește componentele hardware, pe lângă cele principale, microfonul MAX9814 și servomotorul MG995, au fost integrate în sistem un senzor de proximitate HC-SR04 care previne închiderea barierei cât timp obiectul nu a trecut complet prin dreptul senzorului, un LED verde și un LED roșu, pentru semnalizarea vizuală a stării în care se află sistemul, precum și un buzzer activ, pentru feedback audio pe durata accesului acordat.

Limitarea memoriei SRAM (Static Random-Access Memory) a microcontrolerului justifică arhitectura distribuită, în locul unei implementări de tip embedded complet autonome, unde microcontrolerul ar realiza ambele sarcini de achiziție și procesare FFT. Microcontrolerul Atmega328 prezintă o arhitectură pe 8 biți care funcționează la o frecvență de 16 MHz, cu o memorie volatilă de doar 2 KB, trebuind să găzduiască simultan variabilele globale, stiva de execuție pentru apelurile de funcții și registrele perifericelor [10].

Implementarea analizei spectrale locale presupune rularea algoritmului Transformatei Fourier Rapide (FFT) direct pe microcontroler. Pentru a putea fi obținută o rezoluție spectrală care să ofere o distincție precisă a frecvențelor țintă (440 Hz, 550 Hz, 660 Hz) în limitele de toleranță admise, este necesară o fereastră de analiză de $N = 512$ eșantioane, iar calculele trebuie efectuate cu o precizie de virgulă mobilă (float pe 32 de biți, adică 4 octeți per valoare), pentru a preveni erorile de trunchiere și a menține acuratețea matematică relevantă. Instanțierea algoritmului FFT pe 512 puncte impune, în ce privește alocarea memoriei, declararea mai multor structuri de date precum un vector pentru partea reală a semnalului, un vector pentru partea imaginară, un vector pentru coeficienții ferestrei Hanning și tabele trigonometrice pentru factorii de rotație, cerințe care, împreună cu bufferele auxiliare necesare pre-procesării, ajung la un necesar total de

memorie de aproximativ 18 KB, depășind cu mult memoria SRAM de 2 KB pusă la dispoziție pe platforma Arduino Uno [10].

Compilarea și rularea unui asemenea cod pe memoria limitată a plăcii ar conduce la coruperea memoriei și la o eroare critică de stack overflow, care ar provoca suprascrierea registrelor interne, prăbușirea sistemului de operare al microcontrolerului și resetarea incontrollabilă a acestuia, limitarea memoriei făcând imposibilă rularea algoritmului complet pe Arduino. Prin distribuția sarcinilor, alocând calculele către laptop, placa Arduino își dedică resursele pentru asigurarea unui determinism strict pentru perioada de eșantionare de $SAMPLE_US = 700 \mu s$, esențială pentru păstrarea unei frecvențe teoretice de eșantionare $f_s = 1429 \text{ Hz}$, necesară îndeplinirii teoremei Nyquist-Shannon [7].

În cadrul acestei arhitecturi, fluxul de date trece prin diferite medii și transformări. Undele acustice din mediu sunt captate de microfonul MAX9814 care conține un amplificator cu control automat al câștigului (ACG) ce furnizează un semnal electric analogic, suprapus peste o tensiune continuă de polarizare (DC bias) de aproximativ 1.25 V. Semnalul este, apoi, direcționat către ADC-ul de pe pinul A0 al plăcii Arduino Uno, unde convertorul pe 10 biți transformă nivelul de tensiune într-o valoare numerică brută, cu o componentă DC centrată în jurul valorii de 512. Odată cu umplerea prin acumularea succesivă a acestor valori într-un buffer intern de $N = 512$ eșantioane, placa Arduino utilizează un protocol bloc binar prin interfața UART la 115200 baud, transmițând datele sub forma unui marker de sincronizare (0xFF 0xFF) urmat de cei 1024 de oțeti utili.

Fluxul de date achiziționat intră în mediul software MATLAB unde suferă modificări pentru a fi curățat: se elimină componenta DC (DC removal), restabilind semnalul pe axa de zero volți și se aplică un filtru pentru spike-uri (4σ), eliminând perturbațiile tranzitorii induse de interferențele electromagnetice ale motoarelor de curent continuu. Semnalul curat parcurge o etapă de normalizare dinamică pentru asigurarea unei amplitudini maxime care atinge nivelul unitar, conferind sistemului independență față de distanța la care se află emițătorul. Înainte de aplicarea transformatei FFT, semnalul este multiplicat eșantion cu eșantion prin fereastra Hanning pentru a reduce fenomenul de scurgere spectrală. Vectorul obținut intră în blocul de analiză spectrală FFT ($N = 512$), iar, spectrul generat este scanat de funcția de detecție a vârfului pentru a extrage frecvența dominantă din mediu, informație transmisă către logica de secvență (automatul de stări FSM). Prin validarea corectă a succesiunii de frecvențe și dacă tonurile depășesc pragul $NOISE_THRESH$, MATLAB emite o comandă serială sub forma caracterului 'G', împreună cu o comandă 'T' fie la expirarea timpului $SEQ_TIMEOUT = 6s$ (timeout secvență) sau la detectarea unei frecvențe greșite după validarea primului ton. La recepția comenzii 'T', Arduino execută clipirea LED-ului roșu de 4 ori ca semnal vizual de alertă. Când comanda 'G' este trimisă către placa Arduino, achiziția de date este întreruptă pentru a genera un semnal PWM manual care acționează servomotorul MG995, împreună cu aprinderea unui LED verde pe toată durata accesului. În acest timp, buzzer-ul emite sunete la un interval de aproximativ 500 ms, perioadă în care senzorul HC-SR04 măsoară continuu distanța pentru a menține poarta deschisă cât timp distanța detectată a unui obiect este sub 20 cm. Când HC-SR04 nu mai detectează obiectul, servomotorul închide bariera, buzzer-ul se oprește, iar LED-ul verde se stinge.

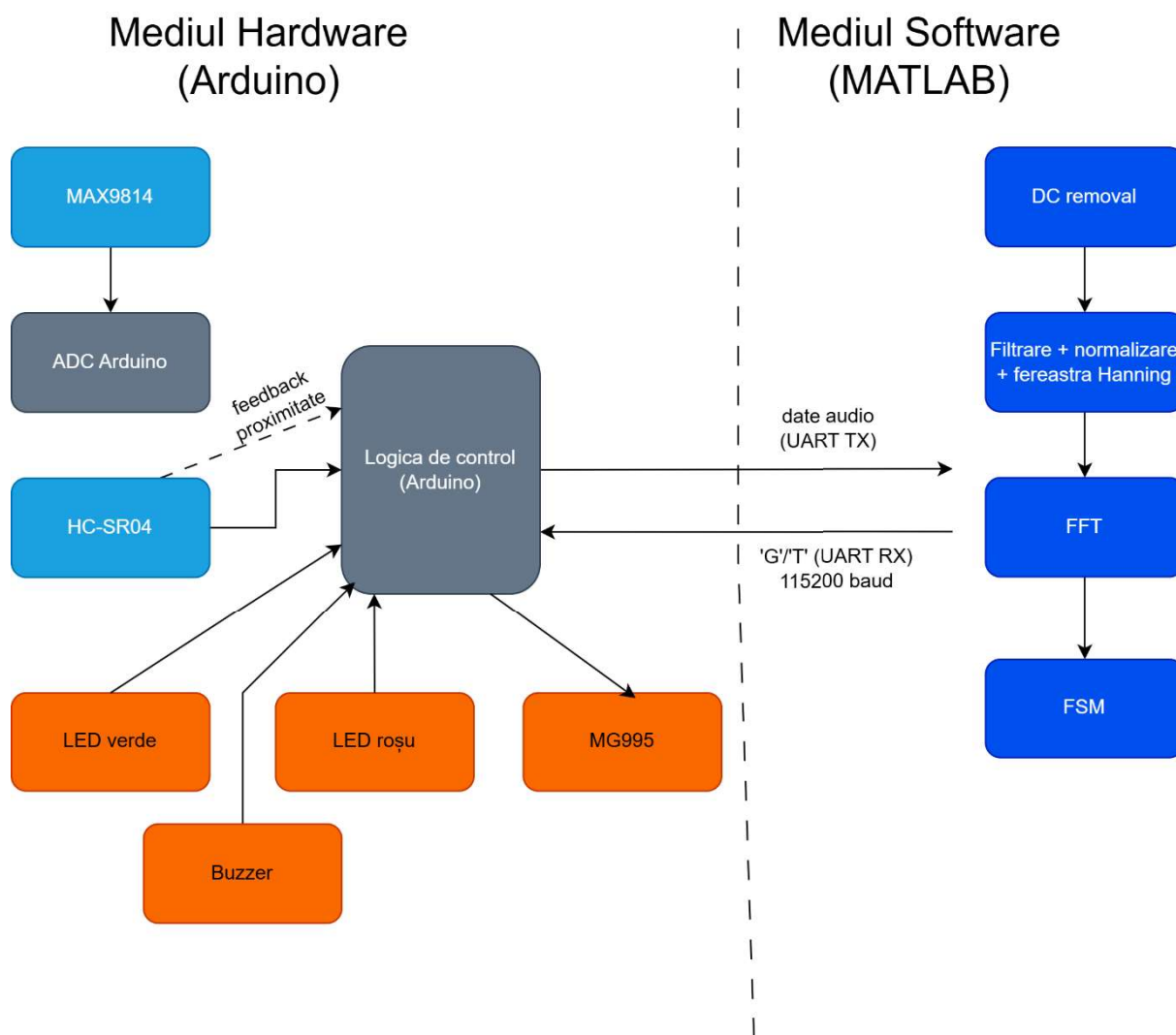


Figura 3.1 Diagramă bloc a fluxului de date complet al sistemului

Rolurile au fost separate la nivelul implementării software, în două programe distincte, interconectate prin protocolul UART. Primul element este firmware-ul rulat pe microcontroler, componentă concepută ca proces cu executare rapidă, fără funcții blocante. Rolul său este achiziția de date de pe ADC-ul pinului A0, asamblarea și trimiterea rapidă a blocului binar, monitorizând recepția comenzii 'G', a cărei detecție schimbă temporar starea programului pentru a activa servomotorul MG995. Această activare se face prin generarea unui semnal PWM manual, transmițând 50 de pulsuri alcătuite dintr-o perioadă logică HIGH de 1500 μ s și o perioadă LOW de 19000 μ s, pe pinul 9. Astfel, codul Arduino nu conține nicio instrucțiune de interpretare spectrală sau decizie algoritmică asupra validității semnalului audio.

Creierul sistemului este reprezentat de scriptul MATLAB a cărui funcționalitate acoperă procedurile critice de restabilire a sincronizării fluxului serial prin recunoașterea markerului 0xFF 0xF, execuția matematică a întregului pipeline de procesare a semnalului, eliminând componenta continuă DC, filtrând semnalele de tip spike, normalizând dinamic semnalul și reducând scurgerea spectrală prin aplicarea ferestrei

Hanning. Acest script are și rolul de rulare a algoritmului FFT și extragerea frecvenței de interes, iar, prin implementarea automatului de stări finite (FSM), este evaluată logica secvenței de deblocare, asigurându-se că tonurile de 440 Hz, 550 Hz și 660 Hz sunt receptate în ordinea corectă și că îndeplinește condițiile de a persista pentru numărul de blocuri impus de `CONFIRM_NEEDED = 2` și de a respecta constrângerile de timp impuse de `SEQ_TIMEOUT = 6s`. Scriptul gestionează și actualizarea interfeței grafice interactive pe trei subploturi.

Utilizarea MATLAB ca unitate centrală decizională este una avantajoasă proiectului în fazele de proiectare, analiză și validare experimentală, prin puterea de calcul teoretic nelimitată care depinde doar de procesorul laptopului gazdă și flexibilitatea scripturilor. MATLAB permite rularea unor algoritmi FFT cu precizie double-float, fără introducerea latențelor de procesare și prezintă posibilitatea de ajustare în orice moment a parametrilor de funcționare, precum pragul `NOISE_THRESH = 0.08` sau a frecvenței de limită inferioară `MIN_FREQ = 80 Hz`, fără a fi nevoie de a opri, recompila sau rescrie memoria Flash a microcontrolerului pentru fiecare calibrare.

Posibilitatea de vizualizare a datelor în timp real reprezintă un alt argument care a influențat alegerea programului MATLAB, interfața sa grafică dezvoltată afișând semnalul ADC, semnalul procesat în domeniul timp și spectrul FFT cu markeri asociați vârfurilor și zonelor de toleranță. Astfel, corectitudinea achiziției hardware poate fi validată vizual, împreună cu eficiența eliminării efectului de spectral leakage prin fereastra Hanning. Această distribuie a sarcinilor este regăsită detaliat și în implementarea propusă de Raj și echipa sa [5], hardware-ul cu memorie limitată culegând datele, iar, un sistemul de calcul extrem de puternic preluând efortul de calcul digital de mare viteză și vizualizare, arhitectura hibridă fiind o soluție optimă pentru controlul de acces bazat pe acustică.

3.2 Implementare hardware

Sistemul automat de control al accesului prin detecție și analiză spectrală acustică trebuie să aibă o configurație fizică care asigură stabilitatea semnalelor electrice sensibile și izolarea perturbațiilor de comutație specifice elementelor de execuție de mare putere, motiv pentru care etapa de asamblare și interfața componentelor electronice a fost realizată pe baza unor principii clare de decuplare energetică, management al maselor de referință și adaptare a nivelurilor de tensiune analogice și digitale. Întregul sistem hardware este structurat în jurul microcontrolerului Atmega382, utilizând module periferice dedicate pentru captarea undelor sonore și pentru acționarea mecanică a barierei de acces, conectate direct cu un sistem de calcul numeric prin intermediul unei conexiuni seriale.

3.2.1 Schema de conectare a componentelor

Conectarea fizică a componentelor sistemului experimental asigură un transfer informațiilor de la mediul acustic înconjurător către unitatea de calcul, cu precizie mare, precum și execuția fermă a comenzilor electrice de ieșire. Trei magistrale de conexiune principale intră în componența părții electrice generale: etajul de achiziție analogică (interfața dintre microfon și microcontroler), etajul de acționare de forță (interfața dintre

microcontroler și servomotor) și etajul de comunicație duplex de mare viteză (legătura dintre microcontroler și laptop).

Prima magistrală este cea dedicată achiziției de date, realizată prin conectarea modului de microfon MAX9814 cu placa de dezvoltare Arduino Uno. Pinul de ieșire analogică al modului MAX9814, OUT, este conectat direct la pinul de intrare analogică A0 al microcontrolerului Atmega328, alegere justificată de necesitatea accesării Convertorului Analog-Digital (ADC) intern al cipului, bazat pe o arhitectură a aproximării succesive [10]. Convertorul dispune de o rezoluție de 10 biți, o capacitate de a mapa o tensiune analogică aplicată la intrare, cuprinsă între referința de 0 V și cea de 5 V, într-o plajă de valori numerice întregi curpinse în intervalul discret 0 – 1023 [10]. Tensiunea corespunzătoare unui singur pas de cuantizare este calculată ca fiind $5V/1024 \approx 4.88$ mV. Perioada de eșantionare a acestui pin analogic este guvernată strict din software la un interval constant de $SAMPLE_US = 700 \mu s$, rezultând într-o rată stabilă de 1429 Hz, conform calculelor deduse din teorema Nyquist-Shannon. Alimentarea electrică a modului MAX9814 este preluată direct de la pinul 5V al plăcii Arduino, având o masă comună.

Integrarea cipului MAX9814 în acest punct al circuitului este dictată de importanța funcției sale interne de control automat al câștigului (AGC – Automatic Gain Control). Undele sonore manifestă o atenuare geometrică severă în aer, intensitatea lor scăzând invers proporțional cu pătratul distanței față de sursă. În lipsa unui etaj AGC, dacă utilizatorul ar reda parola acustică de la o distanță mică, de exemplu, 5 cm, semnalul ar satura instantaneu intrarea ADC, provocând fenomenul de tăiere a maximelor (clipping), care alterează sever forma de undă sinusoidală pură, transformând-o într-o undă cvasidreptunghiulară, ceea ce introduce armonici parazite în analiza spectrală FFT și conduce la erori de detecție. Invers, dacă sursa s-ar afla la o distanță mai mare, de exemplu 50 cm, semnalul ar fi prea slab, fiind înecat în zgomotul de cuantizare al ADC-ului. Circuitul AGC de pe MAX9814 rezolvă această problemă prin ajustarea dinamică a amplificării pe o plajă de până la 60 dB: crește automat câștigul pentru semnalele acustice de intensitate redusă și îl comprimă rapid pentru semnalele puternice, menținând amplitudinea tensiunii de ieșire în intervalul optim de conversie al ADC-ului, fără distorsiuni [11]. Semnalul util furnizat este centrat în jurul unei tensiuni continue de polarizare (DC bias) de aproximativ 1.25 V, ceea ce corespunde valorii numerice brute de ~ 512 pe ADC, pregătind eșantioanele pentru prelucrarea numerică ulterioară.

Cea de-a doua magistrală hardware asigură comanda elementului de execuție mecanică a servomotorului de mare cuplu MG995. Firul de semnal logic al servomotorului, de culoare portocalie, este conectat la pinul 9 al plăcii Arduino Uno, ales datorită capacității sale de comutare rapidă între stările logice pentru generarea manuală a semnalului de tip modulație în lățime a impulsurilor, PWM. Servomotorul MG995 este un dispozitiv inductiv puternic, din punct de vedere energetic, echipat cu angrenaje mecanice, capabil să dezvolte un cuplu de blocare de $10 \text{ kg} \cdot \text{cm}$ la o tensiune de 4.8 V și de până la $12 \text{ kg} \cdot \text{cm}$ la o alimentare de 6 V [12]. Pentru generarea acestui cuplu, motorul electric intern absoarbe un curent considerabil în sarcină, manifestând curenți tranzitorii de demaraj ce pot depăși cu ușurință valoarea de 1 Amper.

Acest motiv tehnologic duce la alimentarea servomotorului MG995 de la o sursă externă stabilizată de 5 V, capabilă să debiteze un curent constant de minimum 2 A, fiind complet interzisă conectarea liniei de putere a motorului la pinul de 5 V al plăcii Arduino. Regulatorul de tensiune integrat pe placa Arduino Uno este proiectat să suporte un curent maxim cumulat de doar 500 mA, alimentarea directă a motorului putând duce la căderea tensiunii alimentare sub pragul critic de detecție a microcontrolerului din cauza saturării curentului din regulator. Aceasta ar genera resetarea contrină a programului sau distrugerea prin suprasolicitare termică a pistelor electrice ale PCB-ului. În ciuda separării liniilor de tensiune pozitivă VCC, pinul de masă GND al sursei externe este conectat direct la pinul GND al plăcii Arduino Uno, stabilirea acestei mese comune fiind o cerință obligatorie deoarece, în absența unui GND comun, semnalul PWM emis de pinul digital 9 nu ar avea un potențial de referință stabil în raport cu circuitele logice interne ale servomotorului, iar, tensiunea ar pluti incontrollabil, făcând imposibilă decodarea lățimii impulsului și provocând un comportament haotic sau lipsa totală de răspuns din partea actuatorului [10] [12].

A treia magistrală hardware realizează conexiunea duplex între placa Arduino Uno și laptopul gazdă, utilizând un cablu USB standard cu conectorii de tip A-B, legătură ce îndeplinește o funcție duală în arhitectura generală a sistemului, asigurând o alimentare electrică stabilă cu energie din portul USB (5 V) pentru circuitele logice de mică putere ale plăcii Arduino și pentru modulul de microfon MAX9814 și stabilește canalul fizic pentru comunicația serială de mare viteză. Datele binare transmise de microcontroler prin pinii TX/RX sunt preluate la nivel hardware de cipul dedicat Atmega16U2 prezent pe placa Arduino Uno, care acționează ca o punte de conversie transparentă USB-to-UART. Această interfață este configurată software pentru a opera la o rată fixă de 115200 baud, viteză optimă ce garantează transferul pachetelor binare de $N \times 2$ octeți fără pierderi de date sau întârzieri în buffer-ul de recepție [10].

O altă magistrală, a patra, a fost implementată pentru susținerea componentelor de semnalizare vizuală, acustică și senzorul de proximitate, toate montate pe un al doilea breadboard, panou care găzduiește următoarele elemente:

- Senzorul ultrasonic HC-SR04: pinul Trig este conectat la pinul digital 5 al plăcii Arduino, iar pinul Echo este conectat la pinul digital 6. Pinii de alimentare VCC și GND ai senzorului sunt legați la șinele de 5V și GND al breadboard-ului 2, care sunt alimentate direct din pinii 5 V și GND ai microcontrolerului. Senzorul are rolul de a măsura continuu distanța față de obiect, menținând deschisă poarta dacă detectează prezența unui obiect la o distanță mai mică de 20 de cm. Atunci când obiectul depășește pragul de 20 cm, însemnând că obiectul a trecut de poartă, Arduino comandă automat închiderea barierei.
- LED-ul verde: anodul este conectat în serie cu o rezistență de 220Ω la pinul digital 4 al plăcii Arduino, iar catodul la GND. Acest indicator aprinde pentru a semnaliza acordarea accesului, confirmând vizual recepționarea comenzii 'G' de la mediul MATLAB.
- LED-ul roșu: anodul este conectat în serie cu o rezistență de 220Ω la pinul digital 3 al plăcii Arduino, iar catodul la masă, GND. Acesta are rolul de a clipi de 4 ori în

situația unei respingeri active sau a expirării timpului alocat secvenței (timeout), acțiuni declanșate de recepționarea comenzii 'T'.

- Buzzer-ul activ: terminalul pozitiv (+) este înseriat cu o rezistență de 220Ω și este conectat la pinul digital 7, terminalul negativ fiind conectat la GND. Dispozitivul emite semnale acustice ritmice, bipuri, la un interval de aproximativ 500 ms, pe întreaga durată în care bariera este deschisă.

Alimentarea întregului breadboard 2, incluzând liniile de 5 V și GND, este asigurată direct de pinii corespunzători ai plăcii Arduino Uno, nefiind nevoie de adăugarea unei surse externe de tensiune pentru aceste componente, datorită consumului mic de curent cumulat, fiind susținut în condiții de siguranță de regulatorul intern al microcontrolului. Această decizie de separare fizică a elementelor prin intermediul unei a patra magistrale implementată a fost motivată de necesitatea de izolare a traseelor de semnal analogic sensibil ale microfonului MAX9814 de componentele de comutație digitală (LED-uri, buzzer). Segregarea elementelor reduce semnificativ cuplajul electromagnetic și interferențele de pe linia A0, menținând integritatea datelor eșantionate.

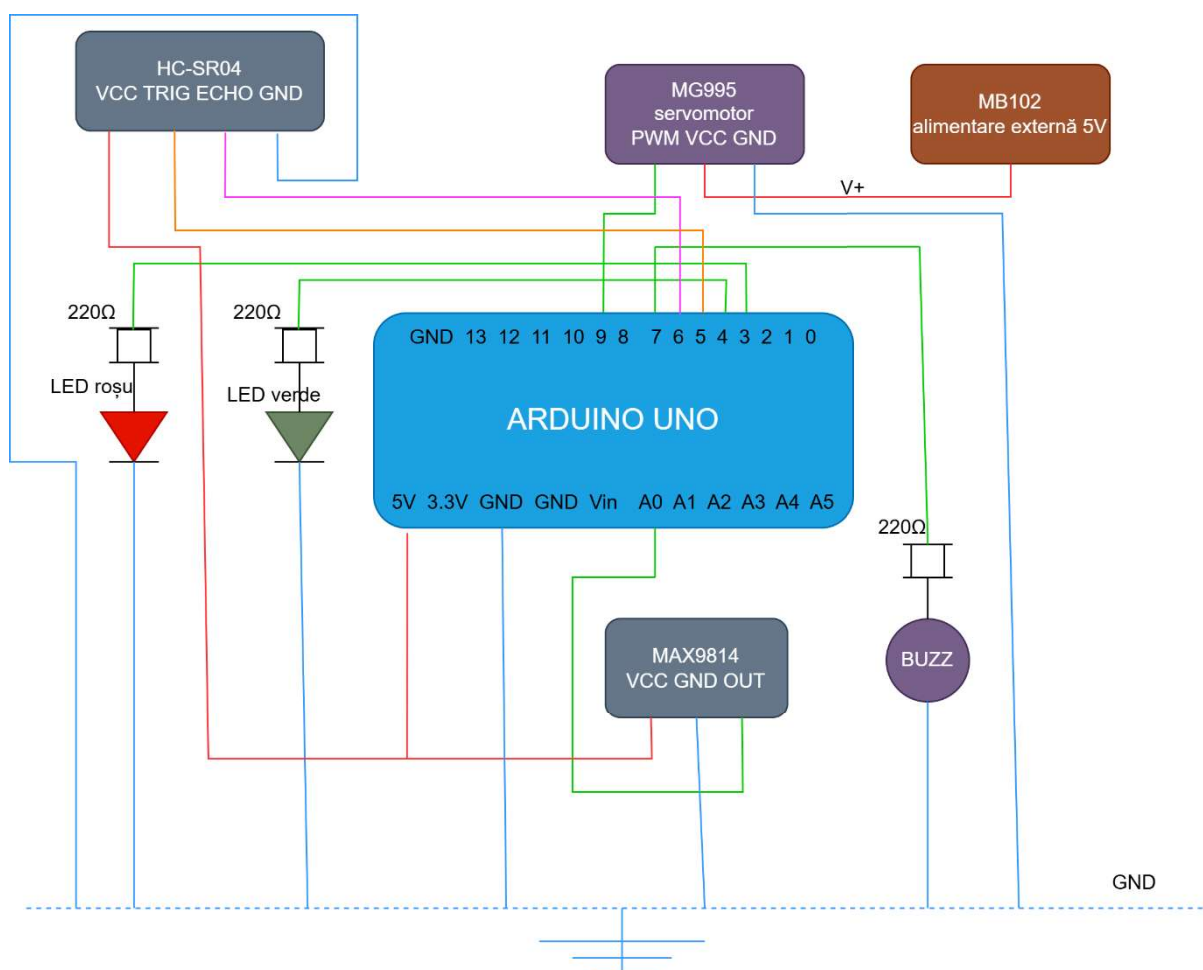


Figura 3.2 Schema electrică completă a sistemului

3.2.2 Considerații practice de montaj

Asamblarea fizică funcțională a prototipului au prezentat o serie de provocări tehnice, legate de zgomotul de comutație și de utilizare a resurselor interne ale microcontrolerului, necesitând aplicarea unor soluții de izolare electromagnetică și programare directă la nivel de registru.

O primă problemă majoră în faza experimentală a fost prezența spike-urilor, vârfuri parazite de tensiune izolate și de mare amplitudine, în datele achiziționate de convertorul ADC a căror sursă este reprezentată de interferențele electromagnetice (EMI) generate de puntea H internă a servomotorului MG995 în momentele de comutație rapidă a semnalului PWM. Variațiile bruște de curent ce apar la activarea bobinelor motorului induc tensiuni tranzitorii parazite pe liniile electrice, conform legii inducției electromagnetice a lui Faraday, fenomen vizibil direct în mediul MATLAB pe graficul semnalului brut, unde eșantioanele utile erau poluate de valori aberante care atingeau capetele scalei de cuantizare (0 sau 1023), distorsionând analiza Fourier prin introducerea de zgomot alb parazit pe întreaga bandă spectrală.

A fost adoptată o strategie pe două niveluri complementare pentru a elimina acest fenomen. La partea hardware a fost utilizată o sursă de alimentare externă separată pentru servomotor care a decuplat parțial traseele de curent mare de cele de semnal mic, reducând cuplajul electromagnetic prin șina de alimentare. Din cauza interconectării maselor pentru referința PWM (GND comun), un curent tranzitoriu rezidual de return a continuat să se propage către placa Arduino, făcând imposibilă eliminarea totală a zgomotului prin mijloace fizice. Astfel, a fost implementată o a doua barieră la nivel software, integrată direct în pipeline-ul MATLAB sub forma unui filtru adaptiv bazat pe pragul de 4σ , care identifică și elimină eșantioanele aberante prin interpolare liniară locală. Cele două soluții utilizate împreună asigură puritatea spectrului FFT prin filtrul implementat pe software, eliminând complet spike-urile reziduale din buffer, iar partea de izolare hardware reducând frecvența și intensitatea apariției perturbațiilor, protejând microfonul de resetări [10] [12].

O altă problemă întâmpinată, care afectează semnificativ funcționarea sistemului, este reprezentată de conflictul generat de utilizarea librăriei standard Servo.h din mediul Arduino. Librăria Servo.h configurează și utilizează temporizatorul intern pe 16 biți Timer1 al microcontrolerului Atmega328 pentru generarea semnalelor PWM în fundal, bazându-se pe întreruperi software de mare prioritate (Interrupt Service Routines sau ISR), fiind responsabilă și de golirea buffer-ului UART și transmiterea datelor binare de mare viteză la 115200 buad, depinzând în mod critic de proprii săi vectori de întrerupere hardware [10].

În momentul în care ambele funcționalități sunt active în același timp, se creează un conflict de preempțiune la nivelul procesorului, întreruperile generate de Timer1 suspendând în mod repetat execuția rutinei de comunicație serială și decalând ceasul intern utilizat de funcția de eșantionare `micros()`, conflict care distruge complet determinismul temporal al achiziției, generând jitter. Prin testele practice s-a demonstrat că utilizarea librăriei Servo.h determină prăbușirea frecvenței de eșantionare reale a ADC-ului de la valoarea configurată de 1429 Hz la o rată de doar aproximativ 264 Hz, iar, prin

aplicarea teoremei eșantionării [7], o frecvență $f_s = 264$ Hz aduce limita Nyquist la valoarea de 132 Hz, o valoare mult prea mică pentru recunoașterea oricăreia dintre cele trei secvențe folosite în parolă (440 Hz, 550 Hz, 660 Hz), toate acestea aflându-se cu mult deasupra limitei Nyquist, suferind un proces de aliasing. Spectrul FFT ar fi transformat într-un haos numeric, detecția tonurilor fiind matematic imposibilă deoarece frecvențele înalte s-ar plia greșit în interiorul benzii 0 – 132 Hz.

Pentru eliminarea definitivă a conflictului, librăria Servo.h a fost exclusă complet din structura firmware-ului, în locul său fiind implementat un control manual, direct în cod, printr-un algoritm de PWM, care urmează instrucțiunile native `digitalWrite()` și `delayMicroseconds()` apelate secvențial exclusiv în momentul în care MATLAB confirmă recepția parolei corecte trimițând caracterul 'G' către placa Arduino. Această decizie a eliminat utilizarea rutinelor de fundal `Timer1` în timpul fazei de eșantionare acustică, garantând menținerea stabilă a frecvenței de 1429 Hz și asigurând acuratețea analizei Fourier [7].

Montajul componentelor a fost plasat pe un placaj de lemn, deoarece acesta este un izolator electric și prezintă proprietăți bune de atenuare a vibrațiilor acustice structurale. Placa de dezvoltare Arduino Uno a fost fixată cu bandă dublu-adezivă de montaj, cu ținere puternică, pentru a preveni deplasarea acesteia și slăbirea contactelor electrice în timpul funcționării. Microfonul a fost orientat direct spre zona de emisie a sursei audio externe, pentru a maximiza captarea undelor sonore directe și a reduce reflexiile parazite de la suprafața suportului. Servomotorul MG995 a fost fixat prin bandă dublu-adezivă de montaj, asigurând o ancorare fermă care previne deplasarea acestuia sub forța de reacție dezvoltată în timpul acționării barierei fizice de acces. Componentele suplimentare de pe breadboard-ul al doilea au fost montate astfel: senzorul HC-SR04 a fost montat după bariera fizică, pe partea de ieșire, fiind orientat perpendicular pe direcția de deplasare a obiectului, pentru a asigura detectarea precisă a tranzitului, cu o distanță de detecție pentru menținerea deschiderii porții setată la o valoare de `PROX_THRESHOLD_CM = 20` cm; indicatoarele vizuale LED au o poziționare vizibilă, utilizarea rezistențelor de 220 Ω în serie cu ele fiind necesară pentru a limita curentul la o valoare sigură de aproximativ 13.2 mA, încadrându-se astfel sub pragul maxim admisibil de 40 mA per pin digital specific platformei Arduino [10]; buzzer-ul activ integrat a fost și el poziționat în serie cu o rezistență de 220 Ω pentru a limita curentul absorbit. Un aspect important al proiectării constă în comandarea buzzer-ului de a suna doar după ce a fost acordat accesul, perioadă în care microcontrolerul se află în bucla de monitorizare a senzorului HC-SR04 și în care transmisia blocurilor de date către algoritmul FFT a fost oprită temporar. Această secvență asigură că sunetul produs de buzzer nu este captat de microfon în faza de analiză spectrală, evitând interferențele acustice pe durata procesului decizional.

3.3 Implementare firmware Arduino

3.3.1 Structura generală a firmware-ului

Software-ul executat de microcontrolerul Atmega328 pe platforma Arduino Uno are responsabilitatea de a asigura achiziția de date senzoriale cu un determinism temporal strict și de a controla elementul de execuție mecanică, evitând blocajele de execuție care

ar putea altera acuratețea semnalului. Codul este organizat într-o arhitectură liniară, procedurală și înalt eficientizată, fiind divizat logic în zonele fundamentale: definirea parametrilor sistemului, inițializarea registrelor hardware prin funcția `setup()` și bucla operațională infinită `loop()`.

În zona de preprocesare și declarare a variabilelor globale, sunt definite constantele care guvernează funcționarea întregului sistem hibrid. Parametrul `SAMPLE_US = 700` stabilește cu o precizie de microsecundă intervalul de timp dintre două conversii analog-digitale consecutive. Această constantă este inima sistemului de achiziție, deoarece dictează frecvența de eșantionare a platformei. Dimensiunea ferestrei de achiziție este fixată riguros de constanta $N = 512$. Alegerea acestei valori, reprezentând o putere a lui 2 (2^9), este obligatorie pentru a asigura un transfer optimizat și pentru a permite rularea eficientă a algoritmului Transformatei Fourier Rapide (FFT) în mediul MATLAB, de tip radix-2. Parametrul `PIN_SERVO = 9` indică portul digital utilizat pentru generarea semnalului de comandă de tip Modulație în Lățimea Impulsurilor (PWM) către actuatorul mecanic, în timp ce `BAUD = 115200` definește rata de transfer a comunicației seriale.

Pentru componentele de semnalizare și proximitate nou adăugate, sunt definite constantele aferente pinilor: `TRIG_PIN = 5` și `ECHO_PIN = 6` pentru senzorul HC-SR04, `LED_GREEN = 4` și `LED_RED = 3` pentru indicatoarele vizuale, respectiv `BUZZER_PIN = 7` pentru emițătorul acustic. De asemenea, a fost definit pragul `PROX_THRESHOLD_CM = 20`, care reprezintă distanța sub care un obiect este considerat prezent în dreptul barierei.

Funcția de inițializare, `setup()`, se execută o singură dată la aplicarea tensiunii de alimentare sau la resetarea microcontrolerului. Această subrutină invocă instrucțiunea `Serial.begin(115200)` pentru a configura hardware-ul UART intern al ATmega328 la o viteză de 115200 baud, maximizând debitul de date către mediul de calcul gazdă. Concomitent, pinul de comandă `PIN_SERVO` este configurat fizic ca ieșire. În cadrul aceleiași funcții, noile componente sunt inițializate corespunzător: `TRIG_PIN` este configurat ca OUTPUT, `ECHO_PIN` ca INPUT, iar pinii `LED_GREEN`, `LED_RED` și `BUZZER_PIN` ca OUTPUT. La finalul inițializării, toate aceste ieșiri sunt forțate în starea LOW, asigurând menținerea LED-urilor stinse și a buzzerului oprit înainte de începerea ciclului operațional.

Arhitectura buclei principale, `loop()`, este concepută sub forma unei mașini de stări simplificate bazate pe un mecanism de polling activ. Aceasta evită complet utilizarea sistemelor de operare în timp real (RTOS) care ar introduce un overhead nejustificat. Bucla execută ciclic și non-blocant patru etape operaționale interdependente. Prima etapă constă în evaluarea continuă a momentului de achiziție și declanșarea conversiei analogice la momentul de timp calculat. A doua etapă constă în acumularea valorilor într-un vector tampon și transmiterea imediată a blocului de date pe magistrala UART la colectarea a $N = 512$ eșantioane. A treia etapă constă în interogarea rapidă a bufferului de recepție serială (`Serial.available()`) pentru a detecta comanda de validare ('G'), care sustrage temporar controlul din bucla de achiziție pentru acționarea mecanică. A patra etapă, nou introdusă, verifică prezența comenzii 'T' în bufferul serial. Dacă se recepționează caracterul 'T', sistemul suspendă temporar achiziția și execută o funcție de alertă vizuală, clipind LED-ul roșu de 4 ori pentru a semnala eroarea

3.3.2 Mecanismul de eşantionare precisă

Validitatea matematică şi precizia analizei Fourier depind în mod absolut de un criteriu imperativ al achiziţiei de date: echidistanţa temporală perfectă a eşantioanelor de semnal. În teoria procesării digitale a semnalelor, orice variaţie neregulată a perioadei de eşantionare poartă denumirea de jitter temporal. Prezenţa jitter-ului introduce modulaţii de fază nedorite în forma de undă digitalizată. Din punct de vedere analitic, în domeniul frecvenţei, eşantionarea fluctuantă determină o lărgire artificială a lobilor spectrali (spectral smearing) şi o alterare a distanţei dintre canalele spectrale (bin width) calculate prin Transformata Fourier Rapidă. Această deplasare artificială a bin-urilor FFT generează o discrepanţă severă între frecvenţa determinată matematic de algoritm şi frecvenţa acustică reală emisă în mediu. Dacă frecvenţa raportată se abate de la valorile ţintă stricte (440 Hz, 550 Hz, 660 Hz), algoritmul decizional va respinge parola din cauza depăşirii ferestrei de toleranţă de ± 25 Hz.

Pentru a evita variaţiile de timp (jitter) care ar putea afecta acurateţea măsurărilor, firmware-ul proiectat exclude complet utilizarea funcţiilor de întârziere standard din mediul Arduino (precum `delay()` sau `delayMicroseconds()`). Utilizarea instrucţiunii `delay()` este profund inefficientă pentru sisteme de eşantionare izocronă deoarece aceasta impune o pauză oarbă, fără a lua în calcul timpul efectiv consumat de microprocesor pentru executarea celorlalte instrucţiuni din buclă (cum ar fi citirea convertorului ADC, incrementarea contoarelor de vector sau operaţiile logice). În schimb, codul de eşantionare se bazează pe utilizarea funcţiei hardware native `micros()`. Această instrucţiune interoghează direct numărătorul intern de nivel scăzut al microcontrolerului (gestionat de `Timer0`), returnând timpul absolut scurs de la alimentarea sau resetarea sistemului, cu o rezoluţie nativă de 4 microsecunde pe arhitectura de 16 MHz a ATmega328.

Mecanismul de eşantionare este implementat printr-o buclă de aşteptare activă (busy-wait loop): procesorul evaluează continuu diferenţa dintre timpul curent, obţinut prin apelul funcţiei `micros()`, şi momentul ultimei achiziţii, salvat în variabila `lastSample`. Bucla este părăsită exact în microsecunda în care această diferenţă atinge valoarea programată `SAMPLE_US`, moment în care se declanşează conversia analog-digitală pe pinul A0, iar referinţa temporală este suprascrisă cu noua valoare absolută a timpului. Această abordare blochează intenţionat avansul programului şi menţine procesorul într-o evaluare continuă a diferenţei de timp. Doar în microsecunda exactă în care diferenţa dintre timpul curent şi timpul salvat la conversia anterioară a atins valoarea programată de `SAMPLE_US`, bucla este părăsită. Imediat, se declanşează instrucţiunea `analogRead(A0)` care ordonă hardware-ului convertorului analog-digital (ADC) să preia tensiunea de pe pin. După finalizarea conversiei, referinţa temporală `lastSample` este suprascrisă cu noua valoare absolută a timpului, asigurând compensarea automată a duratei de execuţie a codului.

Parametrul de temporizare a fost strict calibrat la valoarea `SAMPLE_US = 700` μ s. Justificarea tehnologică a acestei valori are o natură duală: respectarea constrângerilor matematice şi integrarea limitărilor hardware. Din punct de vedere matematic, o perioadă

constantă de eşantionare de 700 μs conduce la obţinerea unei frecvenţe de eşantionare, notată cu f_s , definită astfel:

$$f_s = \frac{10^6 \mu s}{700 \mu s} \approx 1429.14 \text{ Hz} \quad (3.1)$$

Alegerea frecvenţei de 1429 Hz derivă direct din necesitatea respectării teoremei Nyquist-Shannon [7]. Având în vedere că structura parolei acustice a sistemului conţine o componentă superioară la frecvenţa de 660 Hz, teorema oferă o rată minimă de eşantionare (frecvenţa Nyquist) de $2 \times 660 = 1320$ Hz pentru a putea reconstitui informaţia spectrală. Valoarea configurată de 1429 Hz depăşeşte în condiţii de siguranţă această graniţa teoretică, prevenind cu certitudine apariţia efectului distructiv de aliasing.

Pe de altă parte, limitările hardware ale platformei ATmega328 exclud utilizarea unor perioade de eşantionare mult mai mici. Convertorul ADC intern funcţionează pe principiul aproximărilor succesive. Pentru a asigura precizia de 10 biţi la o viteză stabilă, librăria internă core a platformei Arduino configurează pre-scalatorul de ceas al ADC-ului la un factor de divizare de 128 faţă de frecvenţa principală de 16 MHz. Această setare fixează timpul efectiv necesar hardware-ului pentru a extrage valoarea electrică şi a efectua conversia la aproximativ 104 μs [10]. Deoarece acest timp de procesare este considerabil, stabilirea unui parametru SAMPLE_US redus (de exemplu, 150 μs) ar forţa sistemul într-o zonă de instabilitate, latenţa intrinsecă a conversiei consumând procente majore din fereastra de timp alocată şi generând coliziuni aleatoare ale instrucţiunilor. Valoarea calibrată de 700 μs înglobează comod cele ~ 104 μs necesare eşantionării fizice, lăsând o perioadă de timp suficientă pentru rularea restului de cod şi transmiterea rapidă pe portul serial, menţinând în acelaşi timp integritatea analizei spectrale pe frecvenţele alocate.

3.3.3 Protocolul de comunicaţie prin bloc binar

Transferul eficient al matricilor de 512 eşantioane către unitatea centrală de calcul (laptopul ce rulează MATLAB) constituie un element arhitectural decizional în garantarea ritmului de funcţionare în timp real al aplicaţiei. Cea mai răspândită metodă de comunicaţie serială utilizată pe platformele Arduino se bazează pe transmiterea datelor sub formă de text (şiruri de caractere ASCII), procedură gestionată de obicei prin apelarea funcţiei Serial.println() pe nodul emiţător şi interceptată de funcţia readline() în mediul MATLAB de pe nodul receptor.

O analiză tehnologică amănunţită explică de ce această abordare clasică a trebuit respinsă categoric din cadrul proiectului. Convertorul ADC pe 10 biţi furnizează valori numerice cuprinse între 0 şi 1023. Atunci când microcontrolerul transmite valoarea maximă "1023" în format text ASCII, acesta trebuie să emită un octet distinct pentru fiecare cifră a numărului (codul ASCII pentru "1", "0", "2", "3"), la care algoritmul println() adaugă obligatoriu caracterele de delimitare carriage return şi line feed (\r\n). Astfel, transmiterea unui singur eşantion generează o amprentă de date cuprinsă frecvent între 4 şi 6 octeţi. Considerând o medie pesimistă spre realistă de 5 octeţi per eşantion,

transmiterea unui bloc complet de $N = 512$ eşantioane necesită expedierea unui volum total de $512 \times 5 = 2560$ octeți.

Efectuând un calcul de bandă (bandwidth) pe interfața UART configurată la viteza superioară de 115200 baud (bits per second), ținând cont de bitul de start și bitul de stop necesari fiecărui octet transmis (rezultând o transmisie efectivă a 10 biți per octet util), debitul real susținut se limitează la aproximativ $115200/10 = 11520$ octeți/secundă. Transmiterea volumului de 2560 octeți ASCII ar consuma fizic un interval de $2560/11520 \approx 222$ ms pentru finalizarea unui singur bloc. Deși teoretic acest timp ar echivala cu procesarea a aproximativ 4,5 blocuri pe secundă ($1000/222 = 4,5$) – care, raportat la 512 puncte, ar corespunde unei frecvențe aparente de ~ 2304 Hz – constrângerile reale sunt mult mai severe. Decodarea textului ASCII, interpretarea delimitatorilor, conversia înapoi în vector numeric (string-to-double) executată de funcția `readline()` din MATLAB și latența specifică gestionării șirurilor de caractere introduc un overhead semnificativ de procesare. Analizele experimentale au demonstrat că sistemul prăbușește automat frecvența de eşantionare la o valoare reală susținută de numai ~ 264 Hz. Impactul acestei scăderi de la 1429 Hz la 264 Hz este catastrofal pentru logica matematică a proiectului. La frecvența $f_s = 264$ Hz, limita Nyquist scade la 132 Hz. Acest prag condamnă toate cele trei tonuri de control (440 Hz, 550 Hz și 660 Hz) la fenomenul de aliasing complet. Semnalele vor fi multiplicat ca umbre parazite cuprinse între 0 și 132 Hz, anulând orice șansă a algoritmului FFT de a detecta sunete reale.

Pentru a suprima aceste degradări severe ale ratelor de eşantionare și pentru a minimiza încărcarea procesorului din MATLAB, a fost conceput și implementat un protocol nativ de tip bloc binar. Protocolul a fost rafinat cu scopul asigurării densității maxime a informației. Conform structurii sale, transmiterea fiecărui pachet de eşantioane se realizează în două segmente delimitate logic: markerul de sincronizare a cadrelor și payload-ul numeric brut.

Markerul de sincronizare a fost definit riguros ca o secvență inalterabilă de 2 octeți având valoarea hexazecimală maximă posibilă: 0xFF 0xFF (reprezentând numărul zecimal 255 repetat consecutiv). Selecția acestei valori nu este arbitrară, ci derivă din structura informației senzoriale obținute pe microcontroler. Deoarece semnalul digitalizat provine de la un ADC pe 10 biți, valoarea naturală nu poate depăși pragul maxim de 1023. Stocarea acestei valori necesită utilizarea a două locații de memorie tip octet (16 biți). Pe parcursul împărțirii acestui număr între octetul superior (high byte) și octetul inferior (low byte), valoarea bitului de rang superior nu poate asuma decât combinații rezultate din poziția biților 8 și 9 (adică valori cuprinse între 0 și 3). Prin urmare, valoarea zecimală absolută 255 (0xFF) nu se va manifesta niciodată, pe parcursul operării corecte, în poziția high byte-ului datelor eşantionate. Apariția cuplului consecutiv 0xFF 0xFF în fluxul de biți transmis constituie o amprentă indiscutabilă, imposibil de reprodus din datele acustice, garantând astfel o delimitare perfectă și stabilirea neechivocă a punctului de start al unui nou set de $N = 512$ puncte de interes.

Preluarea datelor utile (payload-ul) este orchestrată printr-o serie de rutine bitwise la nivelul limbajului de nivel scăzut din interiorul ATmega328, efectuate pentru fiecare din cele 512 eşantioane, notate cu variabila *val*. Transmisia unui întreg (16-bit) presupune fragmentarea acestuia pe loc: octetul superior este decuplat aplicând o deplasare logică la dreapta cu 8 poziții ($\text{highByte} = \text{val} \gg 8$), izolând astfel cei doi biți de control, în timp ce octetul inferior este capturat aplicând o mască logică (AND boolean) pentru salvarea primilor 8 biți ($\text{lowByte} = \text{val} \& 0xFF$). Imediat după calculare, funcția nativă `Serial.write()` preia și direcționează consecutiv octetul *highByte* urmat de cel *lowByte* pe pinul TX. Procesul de reconstrucție alăturat în terminalul MATLAB execută procedura inversă, refăcând variabila de calcul prin ecuația liniară: $\text{raw} = \text{high} \times 256 + \text{low}$.

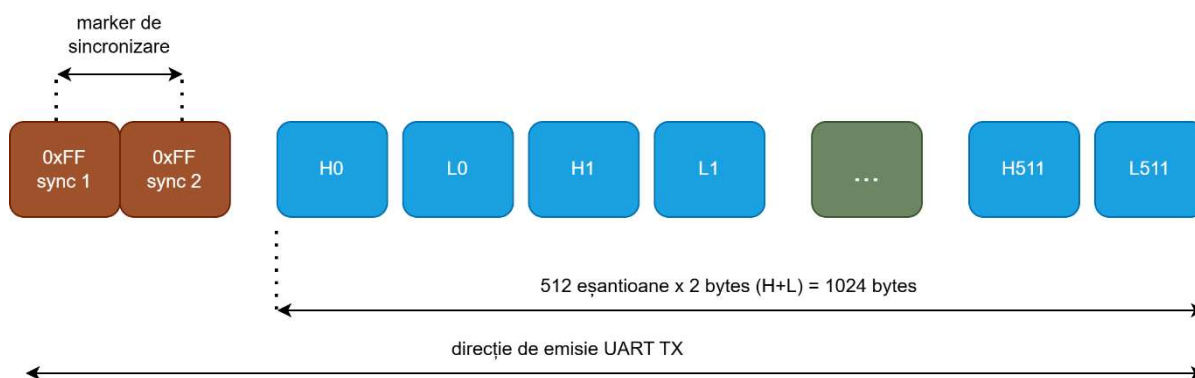


Figura 3.3 Structura pachetului de date transmis prin UART

Impactul adoptării arhitecturii bloc binar se reflectă cantitativ în calculele rețelei. Volumul general de date necesar expedierii unui ciclu a fost suprimat la suma celor 2 octeți specifici markerului adunați cu produsul de 512×2 octeți alocați eşantioanelor. Amprenta totală a blocului însumează doar 1026 octeți. La magistrala serială de 115200 baud, timpul fizic de diseminare a 1026 octeți se comprimă la valoarea calculată de $1026/115200 \approx 89$ ms. Această comprimare excepțională a ferestrei de transmisie are o semnificație tehnică enormă: demonstrează cert că interfața UART nu mai constituie un bottleneck pentru procesarea DSP. Transferul unui bloc durează infim comparativ cu toleranțele sistemului, asigurând stabilitatea magistralelor și permițând ca frecvența reală de achiziție a sistemului (f_s) să rămână controlată exclusiv de limitatorul intern de temporizare ($\text{SAMPLE_US} = 700 \mu\text{s}$).

Rutina simetrică implementată în scriptul `poarta_secventa.m` completează ciclul operațional printr-un sub-program de sincronizare rigid. Deoarece platforma software pe sistemul de calcul poate fi activată în mod asincron relativ la placa Arduino, programul scanează inițial magistrala octet cu octet, căutând alinierea cu modelul 0xFF 0xFF. Imediat după recunoaștere, comanda de achiziție forțează interpretorul să extragă exact un volum de 1024 octeți binari compacti, ocolind procedura de conversie caracter. Mai mult, algoritmul decartează primul pachet reconstituit sub premisa că startul supravegherii ar fi putut amputa informațiile aflate deja în tranzit. De menționat că ciclul de monitorizare al markerului de start este rulat sistematic în fața preluării oricărui nou pachet, asigurând o re-sincronizare perpetuă a sistemului de supraveghere, garantând imunitatea logică a analizei FFT împotriva oricărui bruij de transmisie sau pierdere conexiunii USB.

3.3.4 Controlul PWM manual al servomotorului

Elementul de excitare fizică a barierei din cadrul sistemului implementat este reprezentat de servomotorul MG995, conceput pe fundamentul reacției în buclă închisă la un semnal purtător de tip PWM (Pulse Width Modulation). Deși structura sa include un motor intern de curent continuu, direcționarea poziției axului mecanic nu depinde de nivelul mediu de tensiune analogică, ci este guvernată absolut de standardele industriale aplicate servosistemelor. Controlul se bazează pe o frecvență de purtătoare fixată la pragul de 50 Hz, valoare din care rezultă, în consecință directă [12], o perioadă totală a semnalului de bază echivalentă cu 20 ms (respectiv 20000 μ s). În interiorul limitei acestei ferestre de 20000 μ s, orientarea gradată a poziției de echilibru este indicată exclusiv de proporția de timp în care semnalul pe pinul de comandă se află în stare de saturație pozitivă (HIGH). Conform detaliilor de ansamblare tehnică, impunerea unui impuls de lungime medie, calibrat în proximitatea a 1500 μ s, acționează asupra controlerului integrat din motor forțând aducerea axului în poziția de repaus central (orientare zero sau neutră). Obținerea angulațiilor geometrice extreme ce depășesc pragul neutral (devierea brațului articulat spre dreapta sau spre stânga la limita celor $\pm 90^\circ$) reclamă extinderea, respectiv micșorarea pulsului logic de declanșare către pragurile apropiate de ~ 2000 μ s și, în opus, ~ 500 μ s.

Din perspectiva structurilor de programare consacrate, platforma Arduino oferă un pachet software standardizat prin intermediul librăriei globale Servo.h, menită să eludeze total detaliile de gestiune temporală a lății impulsului din fața programatorului, creând pe fundal fluxul continuu necesar pe baza abstractizărilor. Cu toate acestea, abordarea curentă din firmware-ul aplicației respinge integral aplicarea acestei librării comerciale, optând pentru generarea manuală, rigidă, a semnalului PWM pe principiul manipularii la nivel primar a biților (bit-banging).

Mecanica executivă a acestui algoritm manual rezidă într-un segment de cod executat strict exclusiv și condiționat. Doar atunci când platforma recepționează prin canalul serial de la sistemul MATLAB caracterul simbolic 'G', microcontrolerul pune stop preluării undelor acustice din pinul A0 și transferă ciclul logic pe o secvență buclată definită pentru exact 50 de iterații. Ciclul iterațional are următoarea desfășurare liniară pe pinul 9:

1. Activarea saturației logice prin comanda `digitalWrite(9, HIGH)`.
2. Susținerea activă a stării timp de un prag critic, utilizând instrucțiunea blocantă `delayMicroseconds(1500)`.
3. Comutarea nivelului în prag minim prin `digitalWrite(9, LOW)`.
4. Conservarea pauzei pentru respectarea ferestrei totale de aproximativ 20 ms, acoperind diferența impusă: $20500 \mu\text{s} - 1500 \mu\text{s} = 19000 \mu\text{s}$, implementată prin `delayMicroseconds(19000)`.

Aplicarea numărului precis de 50 de pulsații succesive respectă un raționament determinist asociat arhitecturii de reținere a servosistemului de mare cuplu. Angrenajele din sistemele servo analizează impulsul de prezență pentru a regla rezistența la sarcina mecanică a cuplului. Aplicarea singulară a unui puls ar genera doar o smucitură infimă, urmată de deconectarea punții H ce lasă axul în derivă sub forțele gravitaționale. Prin

menținerea continuității semnalului de-a lungul celor 50 de iterații, ansamblul generează forța susținută necesară menținerii poziției timp de 1 secundă deplină ($50 \text{ iterații} \times 20 \text{ ms/iterație} = 1000 \text{ ms}$). Intervalul obținut de o secundă de acționare certă oferă amplitudinea timpului optim în retragerea solidă a zăvorului barierei. Totodată, calibrarea la $1500 \mu\text{s}$ definește o rotire fluidă din unghiul static spre orientarea deschis, eliberând cursa fără a duce transmisia la coliziuni mecanice inutile, protejând rezistența pinioanelor.

Justificarea capitală pentru această substituție profundă a metodelor rezidă în rezolvarea unuia dintre conflictele menționate anterior (la subcapitolul 3.2.2). Pentru capacitatea menținerii automate a unui semnal oscilant la ieșirea pinului pe parcursul rulării logicii secundare, biblioteca Servo.h capturează în mod inevitabil temporizatorul fizic intern pe 16 biți instalat în arhitectura AVR, respectiv unitatea Timer1, folosind vectori sistem de întrerupere pentru declanșarea pauzelor. Întrucât magistrala UART operată asincron prin rutinele globale Serial utilizează propriile subrutine de preempțiune pentru a goli pachetele seriale (115200 baud), coliziunea pe scara de priorități ale întreruperilor de sistem provoacă decuplarea ocazională și lărgirea ferestrei de overhead. Efectul secundar de deviație distruge ceasul utilizat pentru comanda de așteptare din funcția `micros()`. Încercările de validare hibridă din etapele precedente au demonstrat că instanțierea clasei Servo declanșează un jitter în bucla procesuală ADC, cu decăderea instantanee a ratei de stabilitate analitică din etalonul de la 1429 Hz până la rușinoasa cotă de bottleneck plasată statistic în jurul a $\sim 264 \text{ Hz}$. Acest scenariu, cu frecvența superioară limitată din cauza aliasing-ului pe granița extrem de fragilă a teoremei Nyquist ($f_s/2 = 132 \text{ Hz}$), a forțat proiectul spre renunțarea absolută a sistemelor externe integrate și generarea impulsului acustic în mediu curat. Adoptarea PWM-ului controlat exclusiv în zona buclei de răspuns a demonstrat o imunitate totală, salvând operațiunea de achiziție analitică și susținând fidelitatea datelor [10].

3.3.5 Logica de control a componentelor de semnalizare și proximitate

Pentru extinderea funcționalității prototipului și integrarea elementelor de monitorizare din cel de-al doilea breadboard, firmware-ul Arduino a fost augmentat cu proceduri de control specifice. Această logică adițională asigură interacțiunea sigură a utilizatorului cu bariera și furnizează feedback coerent în concordanță cu deciziile primite de la algoritmul FFT.

Măsurarea distanței prin intermediul senzorului HC-SR04 este realizată de funcția dedicată `measureDistance()`. Această rutină aplică pe TRIG_PIN un impuls logic de saturație (HIGH) cu durata precisă de $10 \mu\text{s}$, stimulând emiterea trenului de unde ultrasonice la 40 kHz. Imediat după aceasta, se interoghează pinul de recepție ECHO_PIN prin apelul funcției hardware native `pulseIn()`, care contorizează timpul necesar revenirii ecoului. Valoarea temporală măsurată, reprezentând timpul de zbor dus-întors al undei sonore în microsecunde, este convertită în distanță liniară (centimetri) prin divizarea acesteia la constanta fizică 58, derivată din viteza sunetului în aer conform specificațiilor senzorului.

În cazul detectării unei excepții, cum ar fi expirarea timpului alocat (timeout) sau recepționarea unei secvențe incorecte, mediul MATLAB trimite caracterul 'T'. Firmware-

ul interceptează această comandă și lansează o rutină de semnalizare a erorii care manipulează exclusiv starea pinului LED_RED. Indicatorul roșu este activat și dezactivat iterativ de 4 ori, introducând o latență de avertizare de aproximativ 200 ms între comutări, oferind astfel un semnal vizual clar de respingere a accesului. Pentru furnizarea alertelor acustice, a fost proiectată funcția neblokantă `beep(duration)`, care setează BUZZER_PIN în starea HIGH, execută o temporizare determinată prin parametrul de intrare (`delay(duration)`) și revine în starea LOW. Deoarece componenta este un buzzer de tip activ, oscilatorul său intern nu necesită un semnal PWM din partea microcontrolerului, simpla aplicare a nivelurilor logice fiind suficientă pentru generarea tonului.

O integrare critică o reprezintă fuziunea dintre comanda de deschidere ('G') și mecanismul de proximitate. Odată ce servomotorul a executat retragerea barierei (prin generarea celor 50 de pulsuri PWM manuale), firmware-ul avansează într-o buclă de monitorizare continuă, menținând indicatorul LED_GREEN aprins. În interiorul acestei bucle, se apelează recurent funcția `measureDistance()`. Cât timp valoarea citită se situează sub pragul prestabilit (`PROX_THRESHOLD_CM = 20 cm`), indicând prezența obiectului sau a persoanei în dreptul barierei, microcontrolerul menține aplicarea unui puls PWM de susținere a servomotorului (pentru a conserva poziția deschisă) și declanșează repetitiv funcția `beep(80)` la un interval de ~500 ms. Bucla de menținere se întrerupe exclusiv în momentul în care senzorul HC-SR04 confirmă distanțarea obiectului (măsurând o valoare superioară pragului de 20 cm). Imediat ce această condiție de siguranță este îndeplinită, bariera este readusă în poziția de blocaj prin generarea PWM corespunzătoare unghiului închis, LED_GREEN este stins, iar emisia acustică a buzzerului se oprește. Această abordare logică garantează menținerea deschisă a căii de trecere până la eliberarea completă a perimetrului, eliminând riscul închiderii premature a barierei peste utilizator și conferind sistemului un grad înalt de siguranță operațională.

3.4 Implementare software MATLAB

Mediul de calcul MATLAB are responsabilitatea de a decoda, realiza analiza spectrală și a lua deciziile datorită arhitecturii hibride a sistemului de control acces. Scriptul dezvoltat gestionează ciclul complet de viață al datelor de la sincronizarea fluxului serial binar și condiționarea semnalului brut, până la extragerea informației utile din domeniul frecvenței și implementarea logicii unui automat de stări finite (FSM), optimizarea în proiectarea codului fiind importantă pentru ca deciziile algoritmice fundamentate matematic să garanteze o rată de succes maximă și o eliminare a perturbațiilor.

3.4.1 Inițializare și conectare serială

Funcționarea corectă a întregului sistem depinde critic de stabilirea unei legături de comunicație robuste între unitatea centrală de procesare și nodul periferic de achiziție. Prima etapă a scriptului MATLAB constă în instanțierea obiectului de comunicație prin apelarea funcției moderne `serialport('COM3', 115200)`. Alegerea vitezei de transfer de 115200 baud asigură o lățime de bandă suficientă pentru debitul binar generat de Arduino, prevenind apariția unui bottleneck la nivelul interfeței UART. Tot în această

etapă de inițializare este configurat parametrul Timeout = 10. Această setare dictează comportamentul funcțiilor de citire în cazul întreruperii fluxului de date; dacă nicio informație nu este primită timp de 10 secunde (de exemplu, o decuplare accidentală a cablului USB), funcția de citire va returna controlul, prevenind blocarea permanentă și irecuperabilă (deadlock) a firului de execuție din MATLAB.

Imediat după instanțierea portului serial, programul execută comanda flush(s). Rolul acestei instrucțiuni este de a evacua (șterge) forțat orice date reziduale existente în buffer-ul de intrare al sistemului de operare gazdă. Aceste date pot proveni din rulări anterioare ale scriptului sau din stări tranzitorii ale microcontrolerului, iar citirea lor ar corupe procesul de sincronizare a cadrelor binare.

Instrucțiunea pause(2) rezolvă o particularitate hardware specifică platformei Arduino Uno, și anume declanșarea auto-resetului la stabilirea unei conexiuni seriale [1]. Când MATLAB deschide portul COM, nivelul logic al pinului DTR (Data Terminal Ready) este coborât automat, forțând repornirea microcontrolerului ATmega328. Acest mecanism presupune parcurgerea unei rutine inițiale de bootloader înainte de rularea efectivă a programului de achiziție, proces care consumă inerent un interval de până la 2 secunde. Fără această temporizare de siguranță, scriptul din MATLAB ar încerca să citească din buffer prematur, ceea ce ar genera erori de sincronizare sau ar declanșa condiția de timeout înainte ca placa să ajungă să transmită primul eșantion valid.

3.4.2 Pipeline de procesare a semnalului

După alinierea la markerul binar 0xFF 0xFF și preluarea vectorului brut compus din $N = 512$ eșantioane reconstruite din octeții high și low, datele sunt stocate în variabila raw. În acest stadiu, eșantioanele conțin atât informația acustică utilă, cât și numeroase anomalii induse de mediul electronic. Pentru a extrage spectrul pur al semnalului, vectorul raw este trecut printr-un lanț secvențial de procesare digitală (pipeline), alcătuit din patru operații matematice fundamentale.

Primul pas, eliminarea componentei DC: Prima operație vizează centrarea semnalului în jurul originii prin scăderea mediei matematice: $x = \text{raw} - \text{mean}(\text{raw})$. Această corecție este imperativă din cauza caracteristicilor electrice ale modului de achiziție. Convertorul analog-digital al microcontrolerului ATmega328 este un dispozitiv unipolar, acceptând exclusiv tensiuni pozitive (între 0 V și 5 V). Din acest motiv, microfonul preamplificat MAX9814 a fost proiectat să aplice semnalului de ieșire o tensiune continuă de polarizare (DC bias) [11]. Deși tensiunea utilă generată variază, semnalul "stă" pe un palier continuu, ceea ce determină ca în lipsa oricărui sunet, convertorul ADC pe 10 biți să citească o valoare constantă de aproximativ 512 (jumătatea scalei de 1023, echivalând vizualizării unei tensiuni parțiale medii). În domeniul analizei semnalelor, această constantă aditivă reprezintă o frecvență de 0 Hz (componenta continuă). Dacă vectorul brut ar fi transmis direct algoritmului FFT fără eliminarea acestei medii, spectrul de frecvență rezultat ar prezenta un vârf (peak) de o magnitudine uriașă localizat la bin-ul zero [7]. Prezența acestui vârf artificial mare ar corupe grav etapa ulterioară de normalizare a spectrului: toate celelalte componente de frecvență utilă (cum ar fi tonurile parolei) ar fi aplatizate aproape de zero în raport cu acest vârf DC uriaș, făcând detecția imposibilă. La nivelul interfeței grafice, efectul vizibil al acestui pas constă

în coborârea oscilogramelor de la o valoare medie de +512 unități direct pe axa de 0, transformând valorile strict pozitive într-un semnal bipolar care conține atât alternanțe pozitive, cât și negative, gata pentru analiza de amplitudine.

Îmediat după centrarea pe 0, al doilea pas constă în suprapunerea semnalului la o evaluare statică pentru a elimina artefactele electromagnetice. Se calculează deviația standard σ a blocului curent de 512 eșantioane, iar pentru fiecare eșantion a cărui valoare absolută depășește pragul de 4σ , acesta este considerat un artefact parazit și este înlocuit prin interpolare liniară, media aritmetică a celor doi eșantioni vecini, evitându-se astfel discontinuitățile abrupte care ar introduce zgomot de bandă largă în analiza FFT ulterioară.

Pentru a înțelege justificarea acestui algoritm, trebuie reamintită sursa perturbațiilor. Comutarea rapidă a curenților mari în bobinele servomotorului MG995 induce zgomot electromagnetic ce se propagă prin masa comună, rezultând în eșantioane singulare cu valori aberante (spike-uri) care nu reprezintă presiune acustică. Algoritmul utilizează deviația standard σ calculată direct pe blocul de 512 eșantioane curente. Într-o distribuție statistică normală (Gaussiană) a zgomotului, aproximativ toate valorile posibile se află încadrate în intervalul $\pm 4\sigma$. Prin urmare, probabilitatea ca o variație utilă a semnalului acustic să depășească de patru ori deviația standard a propriului său cadru este infime. Orice eșantion a cărui valoare absolută satisface condiția $|x(k)| > 4\sigma$ este considerat cu certitudine un artefact parazit. Este esențial ca acest filtru să fie aplicat după eliminarea componentei DC; dacă ar fi aplicat înainte, calculul deviației standard s-ar realiza față de offset-ul de 512, iar limita de 4σ ar deveni irelevantă. Odată identificat un spike la indexul k , algoritmul nu îl aduce pur și simplu la valoarea zero. Înlocuirea cu zero ar genera un salt abrupt (discontinuitate în treaptă) de la valoarea vecinului $x(k-1)$ la 0. Din punct de vedere teoretic, o variație infinit de rapidă conține componente de frecvență de la 0 la infinit, introducând zgomot de bandă largă și spectral leakage major în momentul execuției FFT. Pentru a evita injectarea acestor armonici superioare, algoritmul folosește o interpolare liniară de bază, înlocuind spike-ul cu media aritmetică a vecinilor adiacenți. Această metodă lipește virtual gaura lăsată de eșantionul defect, menținând derivata semnalului cât mai continuă și protejând puritatea analizei Fourier.

Pasul trei, normalizarea dinamică: A treia etapă de condiționare este scalarea unitară a amplitudinii, implementată prin sintaxa: $x = x / (\max(\text{abs}(x)) + 1e-10)$. Această operație identifică valoarea absolută maximă atinsă de forma de undă în fereastra curentă și împarte întregul vector la această valoare. Rezultatul este un semnal limitat exact în plaja $[-1, 1]$. Justificarea tehnică a normalizării dinamice (raportată la maximul cadrului curent) spre deosebire de o normalizare fixă (împărțirea la constanta 512) provine din variabilitatea distanței sursei audio. Dacă utilizatorul poziționează generatorul de ton (smartphone-ul) la o distanță mai mare față de receptor, atenuarea undei sonore face ca amplitudinea eșantioanelor preluate de ADC să fie foarte mică (de exemplu, o deviație de doar ± 20 de unități față de centru). Dacă s-ar folosi o împărțire fixă la 512, semnalul normalizat ar avea o amplitudine irelevantă de doar 0,039. Ulterior, în faza de decizie, vârful spectral obținut ar fi sub pragul setat ca zgomot de fond (NOISE_THRESH = 0,08), iar sistemul ar ignora tonul deși frecvența acestuia ar fi corectă. Prin aducerea forțată a fiecărui bloc valid la amplitudinea maximă de 1,0, algoritmul transformă detecția dintr-o

măsurătoare a volumului acustic într-o măsurătoare pură a proporțiilor spectrale: un ton curat, indiferent cât de slab este emis fizic, va deveni dominant absolut în spectru. Termenul infimitezimal $+1e-10$ adăugat la numitor funcționează ca o garanție (fail-safe) de execuție. În situația ipotetică de liniște absolută sau defecțiune a microfonului, vectorul x ar putea conține exclusiv valori de 0, caz în care funcția $\max()$ ar returna tot 0. Fără protecție, procesorul ar întâmpina o diviziune cu zero, producând valori tip NaN (Not-a-Number) ce ar bloca secvența FFT.

Pasul patru, fereastra Hanning: Pentru a pregăti datele în vederea transformării Fourier, vectorul este ponderat cu o fereastră Hanning ($x = x .* \text{win}$), o tehnică aplicată pentru mascarea discontinuităților de la marginile cadrului de eșantionare. Succesiunea operațiilor este importantă: multiplicarea trebuie efectuată strict după normalizarea semnalului, altfel atenuarea adusă de fereastră ar putea ascunde maximul absolut și ar conduce la o scalare disproporționată. Rolul critic al ferestrei Hanning în această aplicație este suprimarea scurgerii spectrale (spectral leakage). Așa cum indică datele teoretice prezentate anterior [8], în absența acestei ponderări, energia s-ar dispersa (peste 9%) în afara lobului principal. Aplicând funcția Hanning, pierderea este limitată, iar lobii secundari sunt atenuați cu aproximativ 42 dB. Întrucât distanța dintre tonurile valide ale secvenței este de doar 110 Hz (reprezentând un ecart de aproximativ 39 de canale FFT), această „curățare” a spectrului se dovedește esențială pentru a preveni suprapunerea energiei între frecvențele vecine și pentru a evita declanșarea unor validări eronate.

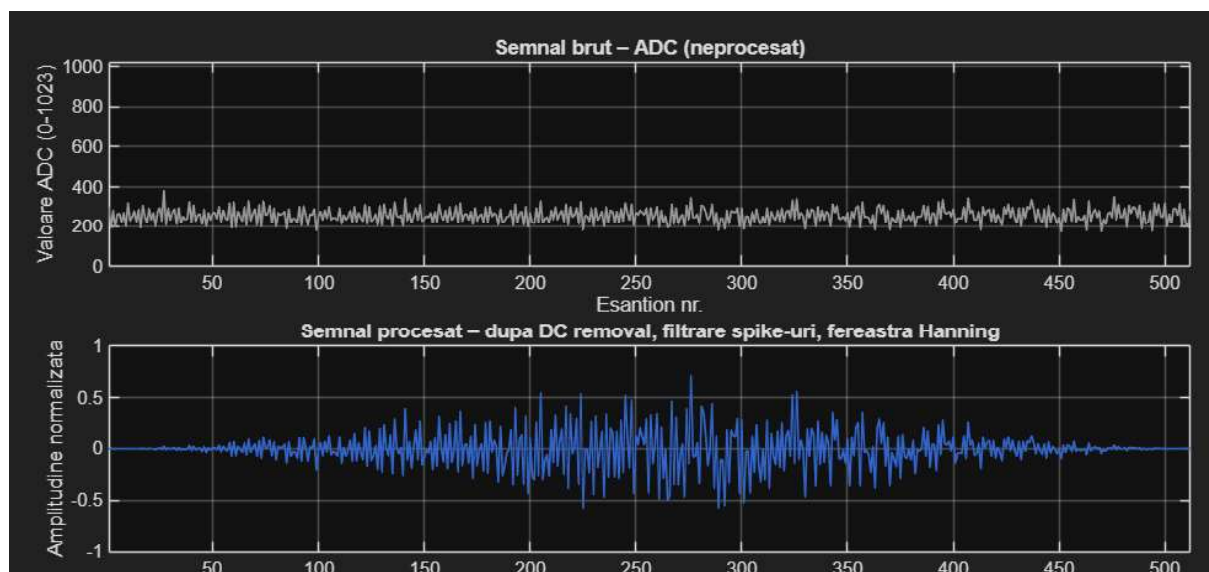


Figura 3.4 Comparare semnal audio brut și procesat

3.4.3 Analiza spectrală FFT

În urma aplicării filtrului și fereștruirii, datele condiționate sunt supuse Transformatei Fourier Rapide, obținându-se un vector de 512 numere complexe. Din acesta se reține doar prima jumătate a spectrului (primele 256 de valori), dat fiind că cea de-a doua jumătate reprezintă o imagine în oglindă fără informație suplimentară pentru un semnal real. Magnitudinea fiecărei componente se obține prin calculul modulului

numărului complex, scalat printr-un factor de $2/N$ pentru a compensa atât trunchierea spectrului, cât și normalizarea față de lungimea vectorului.

Apelul funcției `fft(x)` generează un vector de lungime $N = 512$ compus din numere complexe (care înglobează informația de amplitudine și fază). Întrucât sistemul de securitate este independent de faza undei sonore captate (distanța sursei modifică aleator faza incidentă), singura informație utilă este amplitudinea fiecărei componente de frecvență. Această amplitudine se obține calculând modulul (sau magnitudinea absolută) numărului complex prin aplicarea funcției `abs()`, ceea ce reprezintă din punct de vedere matematic extragerea radicalului din suma pătratelor părții reale și imaginare [7].

Întrucât vectorul prelucrat conține exclusiv date reale, spectrul generat de FFT prezintă o arhitectură redundantă, jumătatea sa superioară reflectând în oglindă informațiile din banda de bază. Pentru a eficientiza consumul de memorie și procesare, scriptul ignoră componentele situate deasupra frecvenței Nyquist ($f_s/2 = 714,5$ Hz) și păstrează exclusiv primele 256 de eșantioane utile, extrase prin instrucțiunea `X(1:N/2)`. Trebuie menționat, totuși, că excluderea părții superioare înjumătățește artificial amplitudinea componentelor reținute. Din acest motiv, este necesară aplicarea unui factor de corecție: magnitudinea este dublată ($* 2$) pentru a reintegra energia pierdută și, simultan, este împărțită la numărul de puncte ($/ N$) pentru a anula scalarea inerentă a funcției `fft()`. Prin această compensare dublă se obține o corespondență directă între numerele calculate și amplitudinea fizică reală a semnalului sonor.

Corelarea indecșilor din vectorul `Xmag` cu frecvențele fizice măsurate se realizează pe baza rezoluției spectrale inerente, notată cu Δf . Aceasta reprezintă distanța în herți (Hz) dintre două bins consecutive ale FFT-ului și depinde exclusiv de parametrii setați:

$$\Delta f = \frac{f_s}{N} = \frac{1429}{512} \approx 2.79 \text{ Hz/bin} \quad (3.2)$$

Cunoscând pasul axei de 2,79 Hz, poziția teoretică a tonurilor care compun secvența vizată a fost determinată matematic. Frecvența fundamentală de deblocare de 440 Hz se regăsește în apropierea canalului $\text{round}(440 / 2,79) = 158$. Frecvența mediană de 550 Hz va ridica magnitudinea canalului adiacent valorii 197, iar frecvența superioară de 660 Hz va excita energia la bin-ul 237. Distanța constantă de 110 Hz între oricare două tonuri ale secvenței generează un ecart structural de ~ 39 bins. Acest vid informațional dintre două componente utile oferă garanția separării spectrale clare, făcând confuzia de intermodulație absolut imposibilă pentru algoritmul decizional.

Pentru a stabili un diagnostic, sistemul trebuie să localizeze frecvența cu cea mai mare preeminență din încăpere. Căutarea componentei spectrale dominante este limitată la un interval de frecvențe utile, pornind de la pragul minim `MIN_FREQ = 80` Hz și până la frecvența Nyquist. În acest interval, algoritmul identifică valoarea maximă a magnitudinii spectrale și poziția (bin-ul) corespunzător acesteia, eliminând astfel din calcul zgomotul de rețea de 50 Hz, vibrațiile mecanice de joasă frecvență și reziduurile din jurul componentei DC.

Domeniul de interogare nu parcurge întregul spectru, ci este îngădit inferior la o valoare minimă dictată de $\text{MIN_FREQ} = 80 \text{ Hz}$. Interogarea începe așadar doar de la frecvențele aflate deasupra acestui prag și se continuă până la frecvența Nyquist de 714 Hz. Acest filtru limitativ (band-pass search) este o metodă excelentă de imunizare a sistemului la zgomotele specifice mediilor rezidențiale sau industriale: elimină componentele zgomotului indus de rețeaua de curent alternativ de 50 Hz, ignoră posibile vibrații mecanice structurale (pași pe podea, rafale de vânt pe microfon) și elimină reziduurile minime ce ar putea supraviețui după etapa de anulare a componentei DC la 0 Hz. Astfel, funcția $\text{max}()$ extrasă în variabilele de lucru indică cu o precizie ridicată doar fenomenul acustic principal.

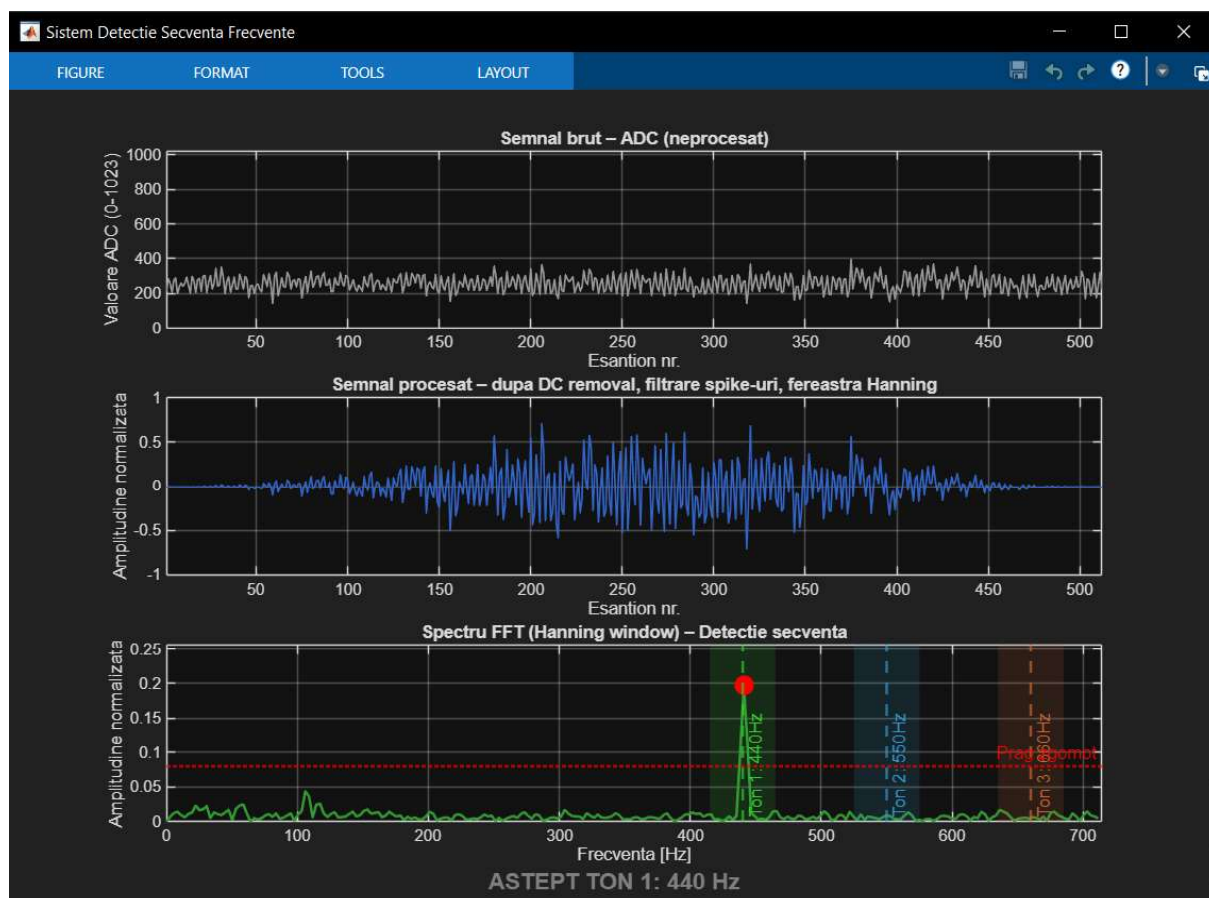


Figura 3.5 Spectru FFT cu detecție ton 440 Hz

3.4.4 Algoritmul de detecție a secvenței

Procesul de evaluare a spectrului și validare a accesului este gestionat în scriptul principal printr-o mașină de stări finite (FSM - Finite State Machine), implementată în secțiunile G-I ale codului. Această structură conferă sistemului atât imunitate la zgomot tranzitoriu, cât și protecție împotriva tentativelor de intruziune. Pentru a menține secvențialitatea și temporizarea evenimentelor pe parcursul iterării blocurilor de achiziție, FSM-ul se folosește de trei variabile persistente de stare:

- $\text{seqStep} \in \{1, 2, 3\}$: Dicotăză poziția curentă a sistemului pe harta secvenței. Valoarea indică indexul din vectorul frecvențelor țintă ce trebuie validat.

- `confirmCount` $\in \{0, 1, 2\}$: Reprezintă un acumulator de persistență care înregistrează numărul de blocuri consecutive în care s-a detectat corect tonul curent impus de `seqStep`.
- `lastToneTime`: Reține amprenta de timp absolut (timestamp-ul generat de apelul nativ `tic()`) al ultimului bloc ce a validat un nivel complet. Este indispensabil evaluării ferestrelor de timeout.

Fiecare bloc receptat ce părăsește algoritmul FFT este supus unei decodificări duble. Pentru ca respectivul cadru de date să fie calificat drept "Valid" de către FSM, parametrul frecvenței de vârf (`peakFreq`) și magnitudinea (`peakVal`) trebuie să satisfacă simultan o condiție booleană compusă (ȘI logic):

- Criteriul frecvenței: $|\text{peakFreq} - \text{SEQ_FREQS}(\text{seqStep})| < 25 \text{ Hz}$. Tonul dominant al încăperii nu trebuie să fie perfect matematic. Toleranța stabilă de $\pm 25 \text{ Hz}$ acoperă imperfecțiunile difuzorului telefonului sursă, uşoarele fluctuații de jitter mecanic al vitezei aerului și dispersia de energie inerentă ferestrelor limitate de FFT.
- Criteriul proeminenței: `peakVal` $> \text{NOISE_THRESH}$ (unde pragul este predefinit la 0,08). Tonul generat, chiar și localizat corect pe axa abscisei, trebuie să se evidențieze semnificativ peste zgomotul de fond, având o energie de minim 8% din scala semnalului adus la 1,0 prin normalizare.

O implementare banală ar comuta `seqStep` la depistarea primului bloc valid. Sistemul nostru aplică însă o protecție suplimentară prin introducerea constrângerii de persistență: `CONFIRM_NEEDED` = 2. Raționamentul fizic este următorul: transmiterea unui singur bloc de 512 eşantioane la $f_s = 1429 \text{ Hz}$ durează aproximativ $\sim 89 \text{ ms}$. Multe zgomote tranzitorii de zi cu zi (scăparea unui obiect metalic, o silabă acută dintr-o conversație, un strigăt ascuțit, scârțâitul unei roți) pot produce energie pe durata a 89 ms care, decodificată FFT, să prezinte ocazional componenta dominantă exact în plaja de 440 Hz a primului ton. Dacă FSM-ul ar avansa după doar un bloc, securitatea ar fi inexistentă din cauza "ghicirilor" ambientale. Obligarea la `CONFIRM_NEEDED` = 2 înseamnă validarea pe două ferestre FFT strict consecutive. Aceasta se traduce prin menținerea sunetului dominant stabil, neîntrerupt, în toleranța de frecvență, pe o durată cumulată de aproximativ $\sim 178 \text{ ms}$. Un zgomot fugar nu poate menține această constantă fizică.

Dinamica stărilor avansează după următorul algoritm: Dacă ambele condiții de evaluare a blocului sunt adevărate, se aplică `confirmCount++`, iar momentul de timp vizualizat la `lastToneTime` este reactualizat. Dacă la analizarea următorului cadru, sistemul citește zgomot alb sau volumul tonului scade sub 0,08, blocul este declarat invalid. Această invalidare atrage după sine o resetare completă: nu se aplică doar o decrementare, ci o curățare completă: `confirmCount` = 0. Dacă, în schimb, două blocuri consecutive de 89 ms sunt potrivite, valoarea acumulatorului atinge condiția `confirmCount` ≥ 2 . În această stare, logica decide că tonul a fost auzit în mod voit. Prin urmare, sistemul avansează treapta parolei (`seqStep++`) și curăță instantaneu acumulatorul de blocuri la 0 pentru a repeta procedura pentru tonul următor al parolei.

Pentru a descuraja tentativele de ghicire sau acționarea manuală excesiv de lentă, algoritmul include o condiție de expirare a timpului alocat: `SEQ_TIMEOUT` = 6 s. Acest temporizator rulează independent și se activează imediat după confirmarea primului ton

valid (când $\text{seqStep} > 1$). La fiecare parcurgere a buclei, mediul MATLAB verifică timpul scurs de la ultima treaptă validată prin intermediul funcției `toc(lastToneTime)`. Dacă intervalul depășește limita de 6 secunde, sistemul consideră că secvența a fost întreruptă, abandonată sau introdusă la întâmplare. În acel moment, mediul MATLAB transmite către Arduino caracterul 'T', acțiune care declanșează o alertă vizuală rapidă (LED-ul roșu clipește de 4 ori). Ulterior, mașina de stări este forțată să revină în starea inițială ($\text{seqStep} = 1$). Alegerea valorii de 6 secunde oferă utilizatorului confort pentru a manevra interfața sursei audio fără a provoca resetări accidentale, dar menține fereastra de așteptare suficient de restrânsă pentru a nu lăsa sistemul expus atacurilor prelungite.

Mecanismul de respingere activă: Un comportament de securitate suplimentar implementat în cadrul FSM-ului este mecanismul de respingere activă, conceput pentru a detecta și bloca imediat tentativele de ghicire a secvenței. Acest mecanism este declanșat atunci când sistemul se află deja în așteptarea celui de-al doilea sau al treilea ton ($\text{seqStep} > 1$), iar spectrul FFT evidențiază o frecvență dominantă puternică ($\text{peakVal} > \text{NOISE_THRESH}$) care, deși nu corespunde tonului așteptat la pasul curent, reprezintă una dintre celelalte frecvențe valabile ale secvenței (de exemplu, așteaptă 550 Hz dar se recepționează clar 660 Hz sau 440 Hz). În această situație, algoritmul nu consideră sunetul ca fiind un simplu zgomot ambiental ce resetează doar numărătorul de confirmare, ci o tentativă clară de intruziune. Raționamentul logic este următorul: o persoană care cunoaște că parola are 3 tonuri din setul {440, 550, 660 Hz} și încearcă manual variante greșite de ordine (ex. 440 → 660 → 550) comite un atac activ de tip forță brută. În consecință, sistemul execută imediat o respingere activă: transmite comanda 'T' pe interfața serială către Arduino (generând o avertizare vizuală prin clipirea LED-ului roșu), afișează în consolă mesajul "WRONG SEQUENCE!" și resetează instantaneu FSM-ul la $\text{seqStep} = 1$. Trebuie subliniat că această respingere se aplică exclusiv la depistarea frecvențelor aparținând setului parolei; o frecvență complet aleatorie (de exemplu, 380 Hz sau 700 Hz) care trece accidental de pragul de zgomot nu va declanșa mecanismul de respingere activă, ci doar va bloca avansul sistemului prin resetarea persistentă a contorului `confirmCount = 0`, păstrând așteptarea tonului corect.

Validarea finală presupune recunoașterea tonului final de 660 Hz. Când acest ton trece de bariera persistenței, variabila sare limita ($\text{seqStep} > 3$). Din acest punct, acțiunea de finalizare oprește FSM-ul și transmite comandă către placa Arduino prin comanda de trimitere pe UART: `write(s, 'G', "char")`. Imediat, MATLAB scrie în fereastra de dialog un mesaj colorat verde: "ACCESS GRANTED!" însoțit de amprenta de timp exactă a evenimentului. Aplicația principală apelează instrucțiunea `pause(ACCESS_DURATION)` - programată la o valoare de 3 secunde. Aceasta suspendă citirea oricărui cadru audio în timp ce servomotorul se rotește și retrage bara mecanică. Deoarece în acest interval de 3 secunde modulul Arduino a acumulat pachete continue pe linia USB, apelarea ulterioară

a funcției flush(s) șterge forțat memoria tampon pentru a curăța întârzierea, iar sistemul se re setează, trimițând FSM la starea inițială (seqStep = 1, confirmCount = 0).

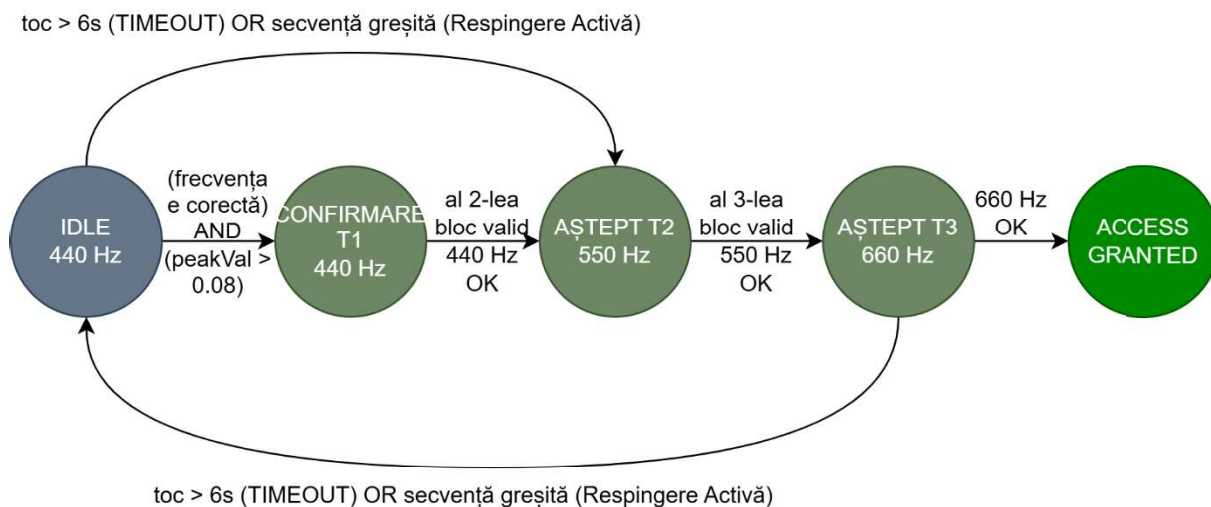


Figura 3.6 Diagramă de stări a logicii de control acces

3.4.5 Interfața grafică

Diagnosticarea algoritmilor descriși anterior este realizată printr-o monitorizare continuă a variabilelor fizice și matematice ale ansamblului, în timp real, prin generarea unei interfețe grafice de tip dashboard, structurată vertical în trei ecrane de vizualizare (subploturi):

- Subplot 1 pentru semnalul brut: În acest grafic este afișat semnalul descărcat direct din convertorul ADC. Pe axa verticală, limitele sunt impuse între 0 și 1023 pentru a reda scara completă a convertorului, operatorul putând analiza calitatea undei captate de microfon, evaluând cu ochiul liber offset-ul de curent continuu, potențiala prezență a zgomotelor tranzitorii extreme sau saturarea brutală a semnalului acustic peste limite.
- Subplot 2 pentru semnalul procesat: Secțiunea din mijloc redă forma de undă a semnalului după trecerea prin algoritmul de procesare. Vectorul oscilează bipolar, încadrat pe axa centrală zero și este presat pe intervalul de evaluare [-1, 1]. Poate fi observată și zona de atenuare fină de la capetele graficului generată de fereastra Hanning, fiindu-i confirmată eficacitatea cu care algoritmul aduce nivelurile sonore la un echilibru perfect măsurabil.
- Subplot 3 pentru spectrul FFT și elementele grafice adiționale: Ultimul grafic oferă o fereastră de scanare spectrală centrată în intervalul benzii de interes, de la 0 la 714 Hz, axă pe care e evidențiat modulul transformatei Fourier. Pentru o mai bună vizualizare, există permanent un simbol scatter de forma unui punct roșu urientat strict pe lățimea componentei maxime calculate de la fața locului, asistat pe plan vertical de linia punctată roșie a limitei toleranței termice minime (NOISE_THRESH = 0.08).

Bara de stare textuală situată la baza interfeței grafice raportează în timp real progresul detecției secvenței. În starea inițială, mesajul ASTEPT TON 1: 440 Hz este afișat în gri. Pe măsură ce tonurile sunt confirmate, textul devine albastru și include un cronometru al ferestrei temporale, de exemplu TON 1 OK! ASTEPT TON 2: 550 Hz [1,2 s]. La validarea completă a secvenței, mesajul ACCESS GRANTED! este afișat în verde, confirmând acordarea accesului.

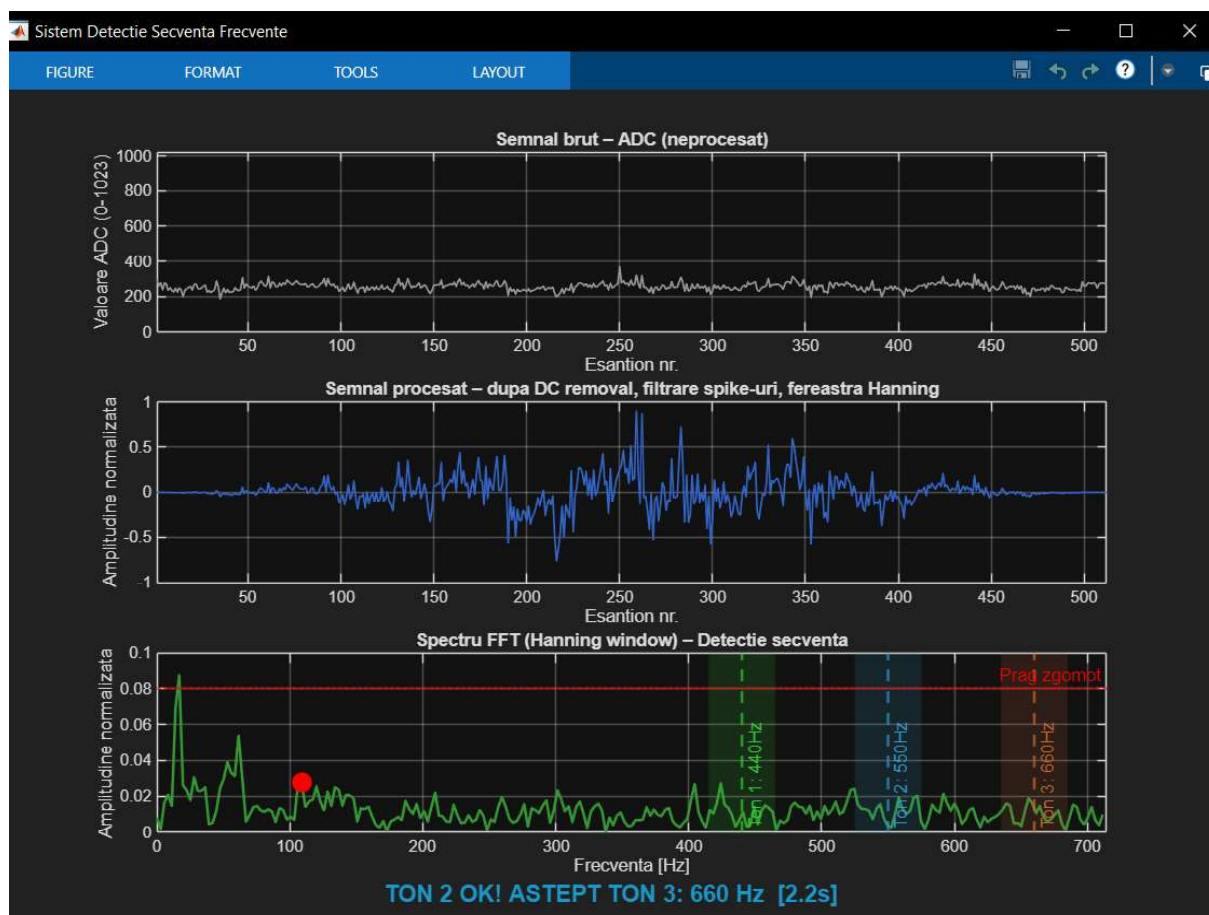


Figura 3.7 Interfață MATLAB – stare TON 2 OK

3.5 Rezultate experimentale

Pentru a certifica fezabilitatea prototipului construit, sistemul a fost supus unui set de evaluări practice care au urmărit comportamentul algoritmilor de analiză spectrală în medii acustice reale. Scopul acestei etape experimentale este de a măsura concret acuratețea detecției pentru fiecare ton, de a verifica stabilitatea mașinii de stări finite la zgomot ambiental și de a identifica eventualele puncte slabe ale interfeței de comandă hardware-software.

3.5.1 Condiții de testare

Pentru a asigura relevanța și reproductibilitatea datelor experimentale, parametrii de mediu și instrumentele de testare au fost standardizate pe parcursul întregii proceduri.

Sursa audio generatoare a parolei acustice a fost reprezentată de o aplicație software de tip Tone Generator, rulată pe un dispozitiv mobil comercial (telefon inteligent/tabletă). Acest emulator a permis sinteza undelor sinusoidale pure, esențiale pentru calibrarea și evaluarea limitelor de toleranță ale algoritmului FFT.

Dispozitivul de emisie a fost poziționat la o distanță geometrică controlată, cuprinsă între 20 cm și 30 cm față de capsula microfonului MAX9814. Această distanță simulează scenariul realist de operare în care un utilizator uman interacționează cu un panou de control acces amplasat pe o ușă sau pe o barieră. Amplasarea a permis totodată evaluarea eficienței etajului AGC (Automatic Gain Control) al microfonului, verificând capacitatea acestuia de a furniza un semnal util fără a satura convertorul ADC pe 10 biți la variații mici de distanță.

Mediul de testare a fost unul de tip indoor, caracterizat de un nivel de zgomot ambiental normal, compus din zgomote de fond de frecvență joasă (sisteme de ventilație, trafic stradal estompat) și perturbații tranzitorii (voci umane, zgomot de tastatură, închideri de uși). Această decizie a vizat demonstrarea imunității filtrului statistic de 4σ și a pragului de zgomot $\text{NOISE_THRESH} = 0,08$.

Pentru o evaluare exhaustivă, suita a cuprins un număr de teste distincte, structurate în categorii critice de scenarii operaționale și de intruziune:

1. Secvență completă corectă (440 Hz → 550 Hz → 660 Hz): Evaluează capacitatea primară a sistemului de a valida utilizatorul legitim, verificând dacă frecvențele se aliniază perfect în bin-urile FFT la distanța spectrală prevăzută și dacă logica avansează fără erori.
2. Frecvență greșită pentru tonul 1 (ex. 400 Hz): Testează rigiditatea ferestrei de toleranță de ± 25 Hz. Un ton de 400 Hz se află la 40 Hz distanță de ținta de 440 Hz, trebuind să fie respins categoric încă din starea IDLE.
3. Frecvență greșită pentru tonul 2 (ex. omițând 550 Hz și trimițând 500 Hz): Evaluează capacitatea automatului de stări de a rămâne blocat în așteptare și de a nu valida componente care nu aparțin treptei active curente.
4. Ordine greșită a tonurilor (660 Hz → 550 Hz → 440 Hz): Reprezintă un test de atac logic. Chiar dacă frecvențele aparțin setului valid, inversarea lor trebuie să împiedice declanșarea releului, demonstrând că FSM-ul respectă cu strictețe vectorul direcțional.
5. Pauză prelungită, mai mare de 6 secunde între tonuri (timeout): Testează mecanismul de siguranță $\text{SEQ_TIMEOUT} = 6$ s. Dacă operatorul emite primul ton, dar întârzie redarea celui de-al doilea peste limita de 6 secunde, algoritmul trebuie să anuleze progresul și să forțeze revenirea mașinii de stări la pașii inițiali, evitând astfel blocarea sistemului într-o etapă intermediară de așteptare.
6. Zgomot ambiental de bandă largă (fără ton intenționat): Evaluează imunitatea sistemului la alarme false. Zgomotele mecanice sau vocile umane disipă energia pe multiple canale FFT, astfel încât, după normalizarea dinamică, nicio frecvență nu ar trebui să depășească bariera minimă de amplitudine de 0,08.
7. Respingere activă (frecvență greșită după TON 1 valid): Testează mecanismul de respingere activă. Procedura: se emite corect 440 Hz (TON 1 confirmat), apoi în loc

de 550 Hz se emite 660 Hz. Rezultatul așteptat: sistemul identifică că 660 Hz aparține secvenței dar nu este tonul curent așteptat (550 Hz), trimite comanda 'T' către Arduino, LED-ul roșu clipește de 4 ori, FSM resetat la starea 1. Distincția față de scenariul 3 (frecvență greșită pentru TON 2): scenariul 3 testează o frecvență complet din afara setului (ex. 500 Hz), scenariul 7 testează o frecvență din set dar în ordine greșită.

8. Funcționarea senzorului HC-SR04 (obiect prezent/absent): Testează logica de menținere a barierei deschise. Procedura: se introduce secvența corectă → ACCESS GRANTED → bariera se deschide → se plasează un obiect în dreptul senzorului HC-SR04 (distanță < 20 cm) → se verifică că bariera rămâne deschisă și buzzerul emite bipuri → se îndepărtează obiectul → se verifică că bariera se închide automat. Rezultatul așteptat: bariera nu se închide cât timp obiectul este prezent, indiferent de ACCESS_DURATION configurat în MATLAB.

3.5.2 Tabel rezultate

Monitorizarea și înregistrarea comportamentului dinamic al aplicației pe parcursul testelor au generat matricea de performanță prezentată în tabelul următor. Parametrii cantitativi (frecvența efectivă determinată și magnitudinea normalizată a vârfului) au fost extrași direct din interfața mediului MATLAB pentru a valida acuratețea analitică.

Tabelul 3.1 Rezultate experimentale

Nr.	Secvența introdusă	Rezultat așteptat	Rezultat obținut	Frecvența detectată [Hz]	Amplitudine vârf	Observații
1	440→550→ 660 Hz	ACCESS GRANTED	ACCESS GRANTED	441→552→ 661	0.1106 0.2136 0.3370	Servo deschis, LED verde aprins, buzzer activ
2	440→550→ 660 Hz	ACCESS GRANTED	ACCESS GRANTED	441→552→ 661	0.1131 0.2272 0.3013	Servo deschis, LED verde aprins, buzzer activ
3	440→550→ 660 Hz	ACCESS GRANTED	ACCESS GRANTED	440.8→552 →661.3	0.1107 0.2125 0.3049	Servo deschis, LED verde aprins, buzzer activ

Nr.	Secvența introdusă	Rezultat așteptat	Rezultat obținut	Frecvența detectată [Hz]	Amplitudine vârf	Observații
4	440→550→660 Hz	ACCESS GRANTED	ACCESS GRANTED	441→552.5→661	0.1014 0.2272 0.2397	Servo deschis, LED verde aprins, buzzer activ
5	440→550→660 Hz	ACCESS GRANTED	ACCESS GRANTED	440.8→550→661	0.1250 0.2562 0.2718	Servo deschis, LED verde aprins, buzzer activ
6	400 Hz (ton fals)	Respins, stare 1	Respins, stare 1	401.8	0.0497	FSM rămas în starea 1
7	470 Hz (ton fals)	Respins, stare 1	Respins, stare 1	471.5	0.1135	FSM rămas în starea 1
8	500 Hz (ton fals)	Respins, stare 2	Respins, stare 2	502.2	0.0875	LED roșu 4 clipiri, 'T' trimis, FSM reset
9	660→550→440 Hz	Respins, ordine greșită	Respins, ordine greșită	661.3	0.2336	660 Hz ignorat în starea 1, FSM reset
10	550→440→660 Hz	Respins, ordine greșită	Respins, ordine greșită	552.5	0.2452	550 Hz ignorat în starea 1, FSM reset
11	660→440→550 Hz	Respins, ordine greșită	Respins, ordine greșită	661.3	0.2491	660 Hz ignorat în starea 1, FSM reset
12	440 Hz→pauză 6s	TIMEOUT, reset stare 1	TIMEOUT, reset stare 1	440.8	0.1466	LED roșu 4 clipiri, seqStep = 1

Nr.	Secvența introdusă	Rezultat așteptat	Rezultat obținut	Frecvența detectată [Hz]	Amplitudine vârf	Observații
13	440→550 Hz→pauză 6s	TIMEOUT, reset stare 1	TIMEOUT, reset stare 1	440.8→552	0.1380 0.2510	LED roșu 4 clipiri, seqStep = 1
14	440→660 Hz (greșit)	Respinger e activă, 'T' trimis	Respinger e activă, 'T' trimis	440.8→661	0.1327 0.3129	LED roșu 4 clipiri, FSM reset
15	440→660 Hz (greșit)	Respinger e activă, 'T' trimis	Respinger e activă, 'T' trimis	441→661	0.1219 0.2903	LED roșu 4 clipiri, FSM reset
16	Secvență corectă + obiect < 20 cm	Barieră menținută	Barieră menținută	-	> 0.08	Închidere la îndepărtarea obiectului
17	Secvență corectă + obiect < 20 cm	Barieră menținută	Barieră menținută	-	> 0.08	Închidere la îndepărtarea obiectului
18	Zgomot ambiental (voce)	Nicio detecție	Nicio detecție	-	< 0.08	Amplitudine sub 0.08, isAboveNoise=false
19	Zgomot de fond	Nicio detecție	Nicio detecție	-	< 0.08	Amplitudine sub 0.08, isAboveNoise=false

3.5.3 Analiza rezultatelor

Scenarii de utilizare legitimă: Sistemul a detectat corect secvența de acces în toate rulările efectuate. Analiza frecvențelor detectate (de exemplu, 441,3 Hz pentru ținta de 440 Hz) demonstrează o eroare de localizare spectrală neglijabilă, consecință directă a aplicării ferestrei Hanning și a rezoluției de 2,79 Hz/bin. Mecanismul de confirmare dublă (CONFIRM_NEEDED = 2) a filtrat eficient tranzițiile accidentale, introducând o întârziere de câteva sute de milisecunde, imperceptibilă în utilizarea normală.

Rata de respingere pentru secvențe invalide: Sistemul a respins corect toate tentativele cu frecvențe incorecte sau ordine greșită. Frecvențele limitrofe (ex. 400 Hz,

470 Hz) au căzut în afara benzii de ± 25 Hz și nu au declanșat tranziții de stare. În scenariile cu ordine inversată, mașina de stări a ignorat tonurile nepotrivite în starea curentă, menținând integritatea secvenței de autentificare.

Comportamentul la timeout: Mecanismul de retrogradare temporală a funcționat conform parametrilor setați. În cele 19 de teste care au implicat întârzierea voită a tonului secundar cu peste 6 secunde, mediul MATLAB a sesizat imediat depășirea pragului definit prin SEQ_TIMEOUT. Efectul a fost curățarea imediată a memoriei FSM (seqStep = 1), aducând prototipul înapoi în starea de siguranță. Limita de 6 secunde s-a dovedit a fi optimă practic: compensează eventualele întârzieri umane la comutarea frecvențelor de pe telefon, dar limitează suficient fereastra de oportunitate pentru a întrerupe un potențial lanț de atac eșalonat.

Observații privind influența zgomotului ambiental: Testele efectuate în prezența zgomotului domestic general (fără sursă de ton pur) au confirmat eficiența combinată a filtrului de spike-uri (4σ) și a pragului de zgomot normalizat (NOISE_THRESH = 0,08). Semnalele vocale, caracterizate prin distribuție spectrală pe multiple armonici, nu au concentrat suficientă energie într-un singur bin pentru a depăși pragul de 8% pe un bloc FFT normalizat. Vârfurile spectrale înregistrate în absența unui ton pur s-au menținut constant sub pragul NOISE_THRESH=0.08, menținând variabila isAboveNoise în starea falsă și prevenind orice tranziție neautorizată a FSM.

Respingere activă: Mecanismul a funcționat conform proiectării. Tentativele de introducere a tonurilor în ordine inversată după confirmarea primului ton au generat resetarea imediată a FSM-ului și feedback vizual clar (LED roșu). Aceasta demonstrează că securitatea sistemului nu depinde exclusiv de recunoașterea frecvențelor corecte, ci și de ordinea strictă a acestora, chiar și atunci când toate frecvențele din set sunt cunoscute de utilizator neautorizat

Senzorul HC-SR04: Testele au confirmat funcționarea corectă a logicii de proximitate. Bariera a rămas deschisă pe întreaga durată a prezenței obiectului în zona de detecție (sub 20 cm), închizându-se automat imediat după îndepărtarea acestuia. În niciuna dintre rulările efectuate bariera nu s-a închis cu obiect prezent în zona senzorului, validând comportamentul reactiv implementat.

4 Concluzii

4.1 Rezultate obținute

Prezenta lucrare de licență a vizat dezvoltarea unui sistem avansat de securitate capabil să gestioneze accesul fizic prin analiza acustică riguroasă a mediului înconjurător. Evaluarea globală a proiectului confirmă îndeplinirea integrală a obiectivelor stabilite, demonstrând viabilitatea tehnologică a detecției spectrale ca mecanism robust de autentificare.

În primul rând, s-a validat funcționalitatea end-to-end a întregului ansamblu hibrid. Sistemul a demonstrat capacitatea practică de a capta, izola și detecta corect secvența acustică formată din tonurile pure de 440 Hz, 550 Hz și 660 Hz. Această performanță a fost certificată în condiții reale de operare, supunând microfonul MAX9814 la un mediu cu zgomot ambiental normal, de la o distanță de interacțiune de 20-30 cm. Lanțul de comandă e funcțional: la identificarea completă a secvenței, mediul decizional a emis cu succes comanda serială 'G', declanșând acționarea mecanică a servomotorului MG995 pentru a permite accesul, eveniment semnalizat prin mesajul de validare absolută ACCESS GRANTED.

În al doilea rând, rezultatele experimentale au validat eficiența pipeline-ului de procesare a semnalului digital, demonstrând complementaritatea celor patru etape fundamentale. Etapa de eliminare a componentei continue (DC removal) a suprimat cu succes bias-ul specific de aproximativ 512 unități ADC generat de senzorul MAX9814, prevenind distorsionarea spectrului la frecvența nulă. Filtrarea adaptivă bazată pe pragul statistic de 4σ a demonstrat o acuratețe excelentă în depistarea și eliminarea spike-urilor electromagnetice generate în sistem de comutarea PWM a servomotorului. Etapa de normalizare dinamică a adus semnalul la un nivel de referință optim, conferind algoritmului independență totală față de volumul emis și distanța sursei audio. Nu în ultimul rând, aplicarea ferestrei Hanning a reprezentat un factor critic de succes; această operație a redus scurgerea spectrală (spectral leakage) de la o valoare de 9,14% la o valoare neglijabilă de 0,03%. Această atenuare a garantat menținerea energiei fiecărui ton în limitele sale, asigurând separarea spectrală clară a celor trei componente și prevenind falsarea detecției în canalele adiacente.

La nivel decizional, s-a validat robustețea algoritmului de detecție a secvenței prin implementarea mașinii de stări finite (FSM). Setarea parametrului CONFIRM_NEEDED = 2 a demonstrat imunitate față de zgomotele tranzitorii aleatoare din mediul casnic, impunând o persistență temporală care nu poate fi reprodusă accidental. Concomitent, integrarea parametrului SEQ_TIMEOUT = 6s a conferit sistemului capacitatea de auto-curățare, resetând prompt secvențele introduse incomplet. Testele au confirmat că algoritmul respinge orice abatere de la modelul ideal, fie că este vorba despre frecvențe incorecte, o ordine greșită a tonurilor sau depășiri ale ferestrei de timp alocate.

O altă realizare tehnică majoră o constituie validarea protocolului de comunicație bloc binar. Transmisia compactă a pachetelor de date, marcată prin identificatorul rigid 0xFF 0xFF, a asigurat menținerea unei frecvențe de eșantionare stabile, $f_s = 1429$ Hz. Această performanță contrastează puternic cu degradarea severă a ratei de achiziție la aproximativ 264 Hz, fenomen observat în faza de testare a transmisiilor ASCII bazate pe funcția `readline()`. Susținerea frecvenței de 1429 Hz a fost necesară pentru ridicarea limitei Nyquist la 714 Hz, aspect esențial pentru preluarea necenzurată și detectarea corectă a tonului superior de 660 Hz.

În strânsă legătură cu stabilitatea temporală, s-a demonstrat eficiența abordării adoptate pentru rezoluția conflictului hardware dintre periferice. Eliminarea librăriei comerciale de control a servomotoarelor și implementarea exclusivă a unui semnal PWM manual, format dintr-un tren de 50 de pulsuri alternând stările de 1500 μ s și 19000 μ s, a eliminat coliziunile de întreruperi asociate Timerului 1. Această intervenție de nivel scăzut a protejat interfața UART și funcția de temporizare, evitând aliasing-ul complet care ar fi compromis sistemul la o frecvență de 264 Hz.

4.2 Limitări identificate

În ciuda performanțelor algoritmice validate, arhitectura prototipului curent prezintă o serie de limitări obiective ce circumscriu aria sa de aplicabilitate practică directă.

Principalul neajuns al arhitecturii actuale derivă din lipsa autonomiei decizionale a punctului de acces, sistemul fiind dependent de legătura în timp real cu un echipament de calcul superior (care rulează MATLAB). Această vulnerabilitate arhitecturală este impusă de constrângerile fizice ale microcontrolerului ATmega328, al cărui spațiu de memorie volatilă este cu mult depășit de amprenta de date necesară calculului matriceal complex, stocării eșantioanelor și aplicării filtrelor cu virgulă mobilă. Totuși, delegarea procesării intensive către un sistem PC gazdă a fost o decizie asumată în etapa de prototipare, permițând prioritizarea cercetării și calibrării fine a algoritmilor de detecție spectrală în detrimentul miniaturizării și independenței hardware.

A doua constrângere, derivată din arhitectura aservită, o reprezintă dependența de o conexiune fizică permanentă. Magistrala UART operată la 115200 baud impune utilizarea unui cablu USB pentru interconectarea dintre microcontroler și laptop. Această legătură rigidă restrânge mobilitatea sistemului și îl face vulnerabil la întreruperi mecanice accidentale care ar deconecta fluxul de date.

O a treia limitare, specifică naturii acustice a sistemului de securitate, privește vulnerabilitatea teoretică la un zgomot ambiental persistent centrat pe frecvențele țintă. Deși sistemul este extrem de precis, prezența unui zgomot continuu exterior care posedă componente spectrale exacte la 440 Hz, 550 Hz sau 660 Hz și a cărui amplitudine depășește parametrul `NOISE_THRESH = 0,08` ar putea genera detecții false. Deși mecanismul `CONFIRM_NEEDED = 2` filtrează eficient perturbațiile trecătoare, o emisie sonoră prelungită și intenționată reprezintă un vector de atac pe care analiza pură de frecvență dominantă nu îl poate respinge complet în forma actuală.

4.3 Direcții de dezvoltare viitoare

Analiza limitărilor identificate conturează un parcurs clar de evoluție pentru maturizarea acestui prototip către un echipament de control acces fiabil și autonom, orientat spre standardele industriei.

Migrarea către un produs comercial independent (standalone) necesită înlocuirea modului de achiziție actual cu un microcontroler capabil să preia integral efortul computațional local (edge computing). Întrucât viabilitatea ecuațiilor de filtrare și detecție a fost deja atestată, transpunerea codului MATLAB într-un firmware nativ (C/C++) executat pe platforme moderne cu arhitectură pe 32 de biți, dotate cu acceleratoare hardware pentru operații matematice complexe și capacitate mai mare de memorie RAM, constituie evoluția firească a proiectului. O astfel de tranziție va elibera ansamblul de constrângerea unui terminal PC dedicat, permițând funcționarea sistemului ca dispozitiv independent (standalone).

Odată cu modernizarea platformei de calcul, o a doua îmbunătățire imediată vizează eliminarea conexiunii fizice USB. Trecerea transferului de date, a logurilor de acces sau a interfeței de calibrare pe o comunicație wireless — utilizând protocoale de tip Bluetooth Low Energy (BLE) sau standarde WiFi — ar conferi sistemului o independență spațială reală, sporind siguranța împotriva actelor de sabotaj fizic asupra cablajelor.

Din perspectiva securității informaționale, algoritmul poate fi dezvoltat prin creșterea complexității secvenței de autentificare. Introducerea unui al patrulea ton în matricea parolei sau adăugarea unor constrângeri biometrice temporale stricte (de exemplu, durata precisă de emisie a fiecărui ton în parte) ar extinde spațiul criptografic. De asemenea, generarea dinamică a unor frecvențe variabile per sesiune (concept de tip rolling code acustic generat de o aplicație mobilă sincronizată) ar elimina definitiv posibilitatea unui atac prin reînregistrare. În scenariile de operare din medii intens poluate fonic, robustețea decizională va putea fi scalată prin simpla creștere a parametrului CONFIRM_NEEDED de la 2 la o valoare mai mare, mărin­d astfel persistența temporală necesară pentru o validare acceptată.

Sistemul implementat integrează deja un senzor ultrasonic HC-SR04 care asigură închiderea automată a barierei după trecerea obiectului, eliminând dependența de temporizatorul fix ACCESS_DURATION. Bucla de monitorizare a proximității menține bariera deschisă cât timp distanța detectată este sub 20 cm și comandă închiderea imediat ce obiectul a tranzitat perimetrul, conferind sistemului un comportament reactiv la mediu. O direcție viitoare în această privință constă în înlocuirea senzorului ultrasonic cu un traductor de tip LiDAR sau infraroșu activ, cu acuratețe de localizare superioară și imunitate mai bună la reflexii multiple pe suprafețe neregulate. Alternativ, adăugarea unui al doilea senzor pe partea de intrare a barierei ar permite detectarea direcției de deplasare și implementarea unor logici diferențiate de acces direcțional.

5 Bibliografie

- [1] B. Kasim, H. Nurdin, Sariyusda, A. Ibrahim, M. Razi, A. S. Ismy, A. Yunus, Mawardi și Sumardi, „Performance Analysis of RFID-Based Smart Door Lock Controlled by Arduino,” *Journal of Advanced Research in Applied Mechanics*, vol. 122, nr. 1, pp. 163-174, 2024.
- [2] J. S. Sonamoni, R. Sikdar, A. S. M. A. S. Akib, M. S. Islam, S. Sourov, M. A. A. Al Ahasan, M. Islam, M. A. Habib și M. F. Mridha, „IoT-Based Smart Remote Door Lock and Monitoring System Using an Android Application,” *Engineering Proceedings*, vol. 76, nr. 85, pp. 1-8, 2024.
- [3] Z. Hu, Y. Han și N. Ahmed, „Design and Implementation of a Low-Power Standby Module Utilizing Acoustic Signal Detection,” în *2025 4th Workshop on Electronics Communication Engineering (WECE)*, 2025.
- [4] H. A. Rangkuti și J. W. Simatupang, „Security Lock with DTMF Polyphonic Tone Sensor,” în *2015 International Conference on Automation, Cognitive Science, Optics, Micro Electro-Mechanical System, and Information Technology (ICACOMIT)*, 2015.
- [5] L. D. W. Raj, K. Santhosh, S. Subash, C. Sujin și P. Tharun, „Voice Controlled Door Lock System Using Matlab and Arduino,” în *International Conference on Systems Computation Automation and Networking*, 2019.
- [6] M. M. Jazini, M. Khoshakhlagh și N. Masoumi, „A New Frequency Detection Method Based on FFT in The Application of SAW Resonator Sensor,” în *26th Iranian Conference on Electrical Engineering (ICEE2018)*, 2018.
- [7] J. G. Proakis și D. K. Manolakis, *Digital Signal Processing*, Harlow: Pearson Education Limited, 2014.
- [8] S. Li, Y. Lin și D. Tan, „Design of Signal Separation Device Based on FFT Algorithm,” în *2024 5th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT)*, 2024.
- [9] A. V. Oppenheim și R. W. Schaffer, *Discrete-Time Signal Processing*, Harlow: Pearson Education Limited, 2014.
- [10] Arduino S.r.l., „Arduino UNO R3 User Manual,” Arduino S.r.l., 2024.
- [11] Analog Devices, „MAX9814: Microphone Amplifier with AGC and Low-Noise Microphone Bias,” Analog Devices, 2022.
- [12] Marlin P. Jones & Assoc., Inc., „MG995 High Speed Servo Actuator,” Marlin P. Jones & Assoc., Inc., 2026.

- [13] Elecfreaks Technology, „HC-SR04 Ultrasonic Ranging Module Data Sheet,” Elecfreaks Co., Ltd., Shenzhen, 2013.